

# Agentic Monte Carlo: Simulating Reinforcement Learning for Black-Box Agents

Dae Yon Hwang<sup>1</sup> Raunaq Suri<sup>1</sup> Valentin Vilecroze<sup>1</sup> Anthony L. Caterini<sup>1</sup>  
Jesse C. Cresswell<sup>1</sup> Noël Vouitsis<sup>1</sup> Brendan Leigh Ross<sup>1</sup>

## Abstract

LLM agents operate in two distinct regimes: open-weight agents amenable to reinforcement learning (RL) and black-box agents whose behaviour must be controlled purely at test time. Although black-box agents are often backed by state-of-the-art proprietary LLMs, API-only access precludes parameter-level optimization, rendering most RL methods inapplicable. To address this limitation, we turn to a known equivalence between RL and Bayesian inference. We propose Agentic Monte Carlo (AMC) to directly sample from the optimal policy of a black-box agent rather than training it through RL. The optimal policy is a posterior over trajectories whose prior we define as the fixed black-box LLM agent. We employ Sequential Monte Carlo to sample from this posterior by learning a value function to steer the agent while leaving the underlying black-box model unchanged. We validate AMC on three diverse environments from the AgentGym benchmark, demonstrating significant improvements over prompting baselines and even outperforming Group Relative Policy Optimization (GRPO) as we scale the test-time compute of our method. AMC demonstrates the feasibility of performing principled RL-style optimization of black-box LLM agents.<sup>1</sup>

## 1. Introduction

Large Language Models (LLMs) have evolved from passive text generators to active autonomous agents capable of multi-step reasoning and environment interaction (Yao et al., 2023; Schick et al., 2023). To operationalize these

<sup>1</sup>Layer 6 AI, Toronto, Canada. Correspondence to: Dae Yon Hwang <daeyon@layer6.ai>, Brendan Leigh Ross <brendan@layer6.ai>.

*Proceedings of the 43<sup>rd</sup> International Conference on Machine Learning*, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

<sup>1</sup>Code is available at <https://github.com/layer6ai-labs/Agentic-Monte-Carlo>

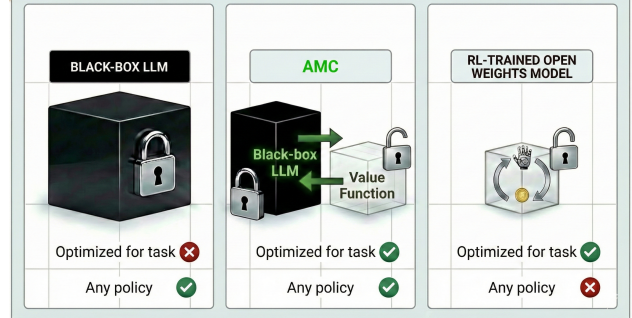


Figure 1. Comparison between AMC and other agentic paradigms. AMC facilitates task-specificity for black-box LLM policies using a learned lightweight value function. On the other hand, training a white-box model with RL imposes constraints on the choice and scale of the base policy.

agents, Reinforcement Learning (RL) (Sutton et al., 1998; Christiano et al., 2017; Ziegler et al., 2019) has become the dominant training paradigm through methods like Proximal Policy Optimization (PPO) (Schulman et al., 2017; Ouyang et al., 2022) and, more recently, Group Relative Policy Optimization (GRPO) (Shao et al., 2024). These algorithms have proven highly effective for open-weight models, enabling abilities ranging from complex mathematical reasoning to software engineering (Wang et al., 2025; Pennino et al., 2025). However, these approaches rely on a fundamental assumption: white-box access to model parameters to compute policy gradients. This requirement presents a fundamental barrier for the most capable state-of-the-art models—such as GPT-5 (OpenAI, 2025b), Gemini 3 (Google, 2025), and Claude 4.6 (Anthropic, 2026)—which are predominantly available only as black-box APIs. Consequently, researchers seeking to optimize agents backed by these proprietary models are often restricted to prompt engineering or fine-tuning open-weight surrogates, neither of which performs RL-based optimization of the target black-box model.

To bridge this gap, we revisit the known duality between RL and Bayesian inference (Levine, 2018; Korbak et al., 2022). This framework treats the goal of maximizing rewards using RL as equivalent to probabilistic inference of the agent’s optimal policy. Here, the optimal policy is defined as a Bayesian posterior distribution over agent trajectories. Specifically, this posterior is proportional to

a prior distribution—given by the pre-trained black-box model—multiplied by a likelihood term representing the probability of achieving high rewards (i.e., of being optimal). This formulation is particularly attractive for black-box agents because it reframes the learning problem: instead of updating the parameters of the prior (which we cannot access), we can simply sample from the posterior distribution to recover optimal behaviour. However, exact sampling from this optimal posterior is impossible for black-box agents, whose full log-probabilities are generally unavailable, and intractable even for white-box agents because of the high-dimensional action space and long time horizons inherent in agentic environments.

In this work, we propose Agentic Monte Carlo (AMC), a novel framework that makes this sampling tractable by leveraging Sequential Monte Carlo (SMC) methods—a form of importance sampling applied to sequential data (Doucet et al., 2001). At each step of the agent’s trajectory, AMC samples actions directly from the black-box prior and reweights them based on expected rewards. To predict expected rewards, we train a separate value function to effectively steer the agent toward optimality without ever modifying the underlying black-box language model. As we highlight in Figure 1, AMC provides a useful interpolation between prompting a static black-box agent and fine-tuning a smaller white-box one.

We validate AMC using the AgentGym benchmark (Xi et al., 2025a), evaluating performance in three diverse environments: WebShop (e-commerce), SciWorld (scientific experiments), and TextCraft (Minecraft-style crafting). Our experiments demonstrate that AMC consistently outperforms prompting baselines and, as we scale the test-time compute of our method, even outperforms GRPO baselines that require full parameter access.

Our key contributions are as follows:

1. We formulate the problem of RL for black-box agents through the lens of Bayesian inference, defining the optimal policy as a Bayesian posterior over trajectories that combines the black-box prior with a likelihood term that encodes optimal agent behaviour.
2. We propose Agentic Monte Carlo, an algorithm that uses SMC with sequential importance resampling to sample from this optimal policy. We derive a weight update rule that leverages a learned value function to guide the black-box agent’s behaviour toward optimality.
3. We empirically validate the effectiveness of our approach compared to both prompting and RL baselines, demonstrating that rigorous RL concepts can be applied to closed-source agents without gradient access.

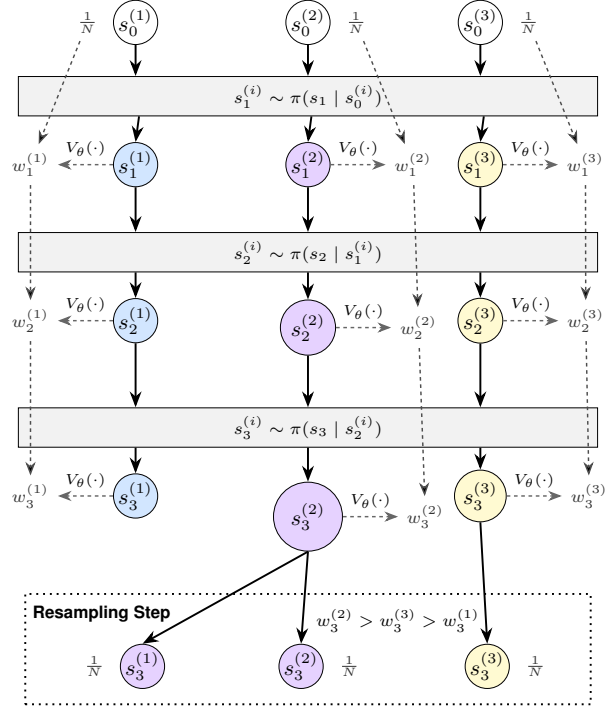


Figure 2. Visual representation of AMC for  $N = 3$  trajectories. Importance weights  $w_t^{(i)}$  are determined using the value function  $V_\theta$ , where lower-weighted trajectories (e.g.,  $s^{(1)}$ ) are more likely to be pruned than higher-weighted ones ( $s^{(2)}$ ,  $s^{(3)}$ ) during resampling.

These results position AMC with black-box models as a viable alternative to gradient-based RL in GPU-constrained settings.

## 2. Agentic Monte Carlo

This section builds up to our method, starting from preliminaries around reinforcement learning. In Section 2.1, we introduce the reinforcement learning setting and formulate it as a Bayesian inference problem. We outline how sequential Monte Carlo (SMC) can be used as a tool to solve this problem in Section 2.2. Then, in Section 2.3 and Section 2.4, we describe Agentic Monte Carlo (AMC): our approach for adapting SMC to potentially black-box agents by learning an ancillary value function.

### 2.1. Reinforcement Learning & Bayesian Inference

We begin our journey in a standard reinforcement learning setting in which an agent, defined by a policy  $\pi$ , responds to its observed state  $s_t \in \mathcal{S}$  at each timestep  $t$  by producing an action  $a_t \sim \pi(a_t | s_t)$ ,  $a_t \in \mathcal{A}$ . The selected action results in a new state  $s_{t+1} \sim p(s_{t+1} | s_t, a_t)$  determined by the transition dynamics of the agent’s environment. We assume discrete state and action spaces. Preferences over states are encoded by a scalar reward function  $r(s_t) \in \mathbb{R}$  given by the environment, where higher rewards are preferred.

Over a time horizon  $T$ , the agent  $\pi$  produces a trajectory  $s_{0:T} \sim \pi(s_{0:T})$ .<sup>2</sup> An optimal agent should produce trajectories that have large cumulative reward values denoted by the shorthand  $r(s_{0:T}) = \sum_{t=0}^T r(s_t)$ .<sup>3</sup>

For many tasks and environments of interest, it has become standard to initialize policies using pre-trained LLMs given their rich semantic understanding and broad world knowledge. Our goal is therefore to fine-tune an LLM agent for a particular environment. Policy gradient methods (Williams, 1992) have become a dominant training paradigm with LLMs: the policy is trained to maximize expected rewards, usually with a KL-divergence penalty term against a reference policy (Ouyang et al., 2022; Shao et al., 2024),

$$\pi_* = \arg \max_{\pi_\theta} \mathbb{E}_{\pi_\theta(s_{0:T})} [r(s_{0:T})] - \beta \text{KL}[\pi_\theta \parallel \pi], \quad (1)$$

where  $\pi_\theta$  is the policy being trained,  $\pi$  is the reference model given by a pre-trained LLM, and  $\beta$  is a regularization coefficient. Although  $\pi_\theta$  is usually initialized from  $\pi$ , the subscript  $\theta$  identifies  $\pi_\theta$  as being a separate model with trainable parameters, while  $\pi$  itself remains fixed.

As highlighted by Korbak et al. (2022) (and for completeness, proven in Appendix A.2), Equation 1 can be understood as performing variational inference to approximate the following (intractable) posterior:

$$\pi_*(s_{0:T}) \propto \pi(s_{0:T}) e^{r(s_{0:T})/\beta}. \quad (2)$$

Equation 2 frames KL-regularized RL as a Bayesian inference problem where the reference policy’s probabilities  $\pi(s_{0:T})$ —which serve as the prior—are modulated to upsample high-reward trajectories as encoded by the likelihood  $e^{r(s_{0:T})/\beta}$ . Note that evaluating the right-hand sides of Equation 1 and Equation 2 yields the same optimal policy: the posterior  $\pi_*$ .

By framing KL-regularized RL as Bayesian inference, we can look beyond variational inference toward the broader Bayesian toolbox. While variational inference (the standard approach) estimates the posterior by training a parameterized policy  $\pi_\theta$ , Monte Carlo methods offer a sampling-based alternative that could bypass policy optimization entirely, allowing us to simulate reinforcement learning even when  $\pi$  is a black box. We exploit this shift in perspective to show that  $\pi_*$  can be estimated by guiding a static prior policy  $\pi$  with a simple auxiliary model. This approach provides a viable pathway for reinforcement learning in scenarios where fine-tuning the reference policy  $\pi$  is intractable or impossible.

<sup>2</sup>Since our policy of interest  $\pi$  will be notionally intractable, we fold transition dynamics into our concept of the policy, ignore actions, and focus on the resulting state transitions  $\pi(s_{t+1} \mid s_t)$  throughout.

<sup>3</sup>We omit discount factors here.

## 2.2. Bayesian Inference via Sequential Monte Carlo

Sequential Monte Carlo (SMC; Gordon et al., 1993; Doucet et al., 2001) is a group of Monte Carlo methods for estimating posteriors of sequentially evolving systems conditioned on step-wise observations. The sequential nature of our state transitions  $\pi(s_{t+1} \mid s_t)$  makes SMC particularly well-suited to the RL setup described in Section 2.1. For example, Piché et al. (2019) use SMC to perform planning with a trained policy and value function, and Zhao et al. (2024) use SMC along with a learned “twist” function for constrained text generation using LLMs. Since the nature of many digital environments allows us to simulate agent trajectories in parallel, we can use SMC to directly structure a Monte Carlo simulation of an agent running the optimal policy  $\pi_*$ .

The most standard SMC method is the bootstrap filter (Gordon et al., 1993; Doucet et al., 2001), i.e., sequential importance resampling (SIR). At a high level, SIR provides a principled way to sample several trajectories  $\{s_{0:T}^{(i)}\}_{i=1}^N$  in parallel by running our (black-box or otherwise intractable) prior  $\pi$ , while intermittently *resampling*: stochastically pruning the worst and proliferating the best trajectories in order to steer the overall sample towards the posterior  $\pi_*$ . Fundamentally, SIR is an importance sampling algorithm, and importance weights will be used to guide these resampling steps.

Consider a *partial* agent trajectory  $s_{0:t}$  up to timestep  $t$ . Since it is intractable to sample trajectories from  $\pi_*$ , importance sampling instead samples from a tractable *proposal* policy and then corrects for the policy mismatch using corresponding importance weights. We know that it is tractable to sample from our (fixed) LLM prior  $\pi$ , so we can sample trajectories  $\{s_{0:t}^{(i)}\}_{i=1}^N \sim \pi$  and then reweight them with importance weights proportional to the ratio of their likelihoods under both  $\pi_*$  and  $\pi$ :

$$w_t = \frac{\pi_*(s_{0:t})}{\pi(s_{0:t})}. \quad (3)$$

In order to use these importance weights for resampling, SIR requires a recursive decomposition of  $w_t$  in terms of the previous weight,  $w_{t-1}$ . In Appendix A.3, we show that

$$\begin{aligned} w_t &= w_{t-1} \cdot \frac{\mathbb{E}_{\pi(s_{t+1:T}|s_t)} [e^{\frac{1}{\beta} \sum_{\tau=t}^T r(s_\tau)}]}{\mathbb{E}_{\pi(s_{t:T}|s_{t-1})} [e^{\frac{1}{\beta} \sum_{\tau=t}^T r(s_\tau)}]}, \\ &= w_{t-1} \cdot e^{V(s_t) - V(s_{t-1}) + \frac{r(s_t)}{\beta}}, \end{aligned} \quad (4)$$

where we define  $V(s_t) = \log \mathbb{E}_{\pi(s_{t+1:T}|s_t)} [e^{\frac{1}{\beta} \sum_{\tau=t}^T r(s_\tau)}]$ .  $V$  is known as the *soft value function* in maximum entropy RL because its log-sum-exp structure emulates a soft maximum reward value over trajectories (Haarnoja et al., 2017; 2018). We refer the reader to Levine (2018) for more background. For now, let us assume that we can compute  $V$ . This recursive decomposition allows us to

perform SIR (Doucet et al., 2001), which we depict visually in Figure 2. The full algorithm, Algorithm 1, is relegated to Appendix A.1 due to space constraints.

SIR runs  $N$  trajectories in parallel and, at each step  $t$ , updates their respective importance weights  $w_t^{(i)}$ . At certain timesteps, we *resample*, with replacement, a new subset of trajectories  $\{s_{0:t}^{(i)}\}_{i=1}^N$  according to their corresponding importance weights  $\{w_t^{(i)}\}_{i=1}^N$ , and then reset their importance weights to be uniform. The decision of whether to resample at a given timestep is made by an arbitrary boolean resampling criterion  $c(t, \{w_t^{(i)}\}_{i=1}^N)$ , which in experiments we simply set to `true` only on a cross-validated subset of timesteps. Resampling criteria are further explored in Appendix D.1.

Upon completion of SIR, the weighted empirical measure of the resulting trajectories  $\{s_{0:T}^{(i)}\}_{i=1}^N$  and weights  $\{w_T^{(i)}\}_{i=1}^N$  converges weakly to  $\pi_*$  as  $N \rightarrow \infty$  (Doucet et al., 2001). Put more simply, sampling from the categorical distribution induced by the normalized weights approximates a sample from  $\pi_*$ , and does so better as  $N$  gets larger.

Until now, we assumed that we could compute the value function  $V$ , which is an expectation involving future rewards  $\{r(s_\tau)\}_{\tau=t}^T$ . Estimating this expectation precisely would require multiple simulations of our agent until the terminal timestep  $T$ , which can quickly become computationally prohibitive. Instead, we opt to *learn*  $V$ , a procedure we describe in the following section.

### 2.3. Learning Importance Weights for Agentic Systems

We now shift our focus to estimating the value function  $V$  to then efficiently compute importance weights when running SIR. We aim to learn a model  $V_\theta$  to approximate  $V$ . Because  $\pi$  is a frozen prior, we can regress our value function directly on a set of Monte Carlo trajectories from  $\pi$ , optimizing for the true value of  $V$  by way of the following relationship. We assume there exists a set of weights for  $V_\theta$  that gives rise to  $V$  as

$$V = \arg \min_{V_\theta} \mathbb{E}_{p(t, s_t)} \left\| V_\theta(s_t) - \log \mathbb{E}_{\pi(s_{t+1:T}|s_t)} \left[ e^{\frac{1}{\beta} \sum_{\tau=t}^T r(s_\tau)} \right] \right\|_2^2, \quad (5)$$

where  $p(t, s_t)$ , the distribution of training inputs, is any distribution with full support over timesteps  $t$  and states  $s_t$ . We can then optimize using gradient-based methods.

In our setting,  $r(s_t)$  is available at test time, so we only need to learn future rewards. We thus parameterize  $V_\theta$  as

$$V_\theta(s_t) = f_\theta(s_t) + r(s_t), \quad (6)$$

where  $f_\theta$  is a transformer-based model with a regression head that outputs a scalar prediction of  $r(s_{t+1:T})$  (if  $r$  were not available at test-time, we would parameterize  $V_\theta$

directly). In practice, we estimate the outer expectation by first sampling  $M$  trajectories  $\{s_{0:T}^{(j)}\}_{j=1}^M$  from  $\pi$  and, from these trajectories, collating a subset of individual states and corresponding timesteps to use as our training set  $\{(t_k, s_{t_k}^{(k)})\}_{k=1}^P$ . We approximate the inner expectation by a single trajectory, thus accepting some bias through the logarithm to arrive at an estimator for the (non-soft) value function as the regression target:

$$\mathcal{L}(f_\theta) = \frac{1}{P} \sum_{k=1}^P \left\| f_\theta(s_{t_k}^{(k)}) - \sum_{\tau=t_k+1}^T r(s_\tau^{(k)}) \right\|_2^2, \quad (7)$$

where we set  $\beta = 1$  for training and adjust the sampling temperature post hoc. It is with this trained network that we model  $V_\theta$ , which then estimates the weight updates needed for SIR in Algorithm 1. In Appendix D.2, we show that training with Equation 7 performs well in comparison to the more expensive and complex Equation 5.

### 2.4. Agentic Monte Carlo

To summarize our method—which we refer to as Agentic Monte Carlo (AMC)—we begin with a black-box (or otherwise intractable) LLM policy  $\pi$  that forms a presumably good prior for a given task and environment of interest. This is common, for example, with today’s state-of-the-art, generalist models accessible only through APIs (e.g., GPT-5, Gemini 3, Claude 4.6), for which we are limited to prompting methods to steer the agent’s behaviour.

Ideally, we would like to be able to fine-tune these agents using standard RL algorithms to estimate  $\pi_*$  from Equation 1, but this feat is impossible due to their black-box nature. Instead, AMC turns to probabilistic inference to approximate  $\pi_*$  using SMC.

Before running SMC, we must build our value function. We collect a series of trial trajectories from the prior  $\{s_{0:T}^{(j)}\}_{j=1}^M \sim \pi(s_{0:T})$  and use these trajectories to train  $f_\theta$  according to Equation 7. We parameterize our value function  $V_\theta$  according to Equation 6.

We next use SMC, depicted in Figure 2, to generate an approximate set of samples from  $\pi_*$  using Algorithm 1, where we substitute the true value function  $V$  with our learned value function  $V_\theta$  in the importance weight computation step. This gives us a final set of trajectories  $\{s_{0:T}^{(i)}\}_{i=1}^N$  and weights  $\{w_T^{(i)}\}_{i=1}^N$  that together approximate  $\pi_*$ . In practice—and in our experiments—we may only be interested in selecting a single best trajectory. We can simply keep the trajectory with the highest cumulative reward or, if  $r$  is unavailable at test time, the trajectory  $i$  with the largest weight  $w_T^{(i)}$ .

### 3. Related Work

**Bayesian Inference, Control-as-Inference, and SMC.** The connections we exploit between KL-regularized RL and Bayesian inference (Korbak et al., 2022) are grounded in the control-as-inference framework (Dayan & Hinton, 1997; Toussaint & Storkey, 2006; Levine, 2018), which casts optimal decision-making as probabilistic inference. Piché et al. (2019) pioneered the use of SMC for planning in this domain, training policy proposals to approximate the optimal posterior. Building on this, Lioutas et al. (2023) introduced CriticSMC, identifying that standard SMC suffers from sample impoverishment in sparse-reward settings; they proposed weighting particles using a learned soft Q-function to steer sampling toward high-value regions. Although AMC shares theoretical roots with these two works, they are designed for continuous control tasks and deterministic transition dynamics, respectively. AMC adapts these principles to the discrete, semantic state space of black-box LLM agents, where gradients are inaccessible and state transitions are defined by opaque—possibly stochastic—tool interactions, rather than physics engines.

**SMC for Language Models.** Recent research has begun applying SMC to steer LLM generation toward desired constraints (Lew et al., 2023; Zhao et al., 2024; Loula et al., 2025). However, these methods modulate the proposal distribution via access to the LLM’s logits, a requirement that is impossible to satisfy with proprietary black-box models; in contrast, AMC is designed specifically for black-box settings. Furthermore, while these works focus on constrained text generation, AMC targets sequential agentic workflows involving external tool interaction. Closer to our work is “Rollout Roulette” by Puri et al. (2025), which applies SMC using pre-trained Process Reward Models (PRMs) (Lightman et al., 2024) to scale test-time reasoning. Instead, we focus on multi-step, interactive agents where the state includes external environment observations, necessitating a critic trained on interaction history rather than a standard reasoning PRM. Furthermore, AMC integrates rigorous RL concepts, viewing the critic not just as a heuristic verifier but as a learned proxy for the (soft) value function in a control-as-inference formulation.

**Search and Planning for LLM Agents.** The limitations of greedy prompting strategies for LLM agents like ReAct (Yao et al., 2023) have spurred interest in inference-time search. Tree-based methods such as Language Agent Tree Search (Zhou et al., 2024) and ExACT (Yu et al., 2025) adapt Monte Carlo Tree Search (MCTS) (Silver et al., 2017) to LLMs but often suffer from high latency due to sequential node expansion. Population-based methods offer a parallelizable alternative. Klein et al. (2025) introduced Fleet of Agents (FoA), which employs a genetic-style particle filter to coordinate agent problem-solving. However, it relies

on static, heuristic value functions (e.g., lexical overlap or prompting). Conversely, AMC integrates rigorous RL by training a parameterized value function to estimate the likelihood of agent success, allowing our method to learn optimality from data rather than relying on hand-crafted heuristics.

### 4. Experiments

We first detail our experimental configuration, and follow that by presenting overall results demonstrating AMC’s effectiveness across benchmarks compared to baselines. We then provide a multi-perspective analysis, including a head-to-head comparison with policy gradient RL, an ablation study on value function optimization, and a cost-efficiency evaluation. Additional analyses, including the impact of the value function’s base model, resampling strategies, metrics, and additional datasets, can be found in Appendix D. Qualitative examples are provided in Appendix E.

#### 4.1. Experimental Configuration

Here we describe our main experimental settings. We relegate additional details to Appendix B and Appendix C.

**Datasets.** Our core evaluations are done across three agentic benchmarks: web-based commerce (WebShop; Yao et al., 2022), scientific reasoning (SciWorld; Wang et al., 2022), and hierarchical crafting (TextCraft; Prasad et al., 2024). To demonstrate the generalizability of our approach, we provide further, albeit limited, evaluation on the Weather and Movie datasets (Ma et al., 2024), relegating these to Table 13 in Appendix D.6. These environments encompass both dense and sparse reward structures. We evaluate our agents using AgentGym (Xi et al., 2025a), a centralized platform comprising a heterogeneous collection of environments that span a diverse range of real-world tasks.

**Training.** To train our value functions, we curate a set of trajectories  $s_{0:T} \sim \pi(s_{0:T})$  on a set of training tasks for each benchmark. We initialize  $f_\theta$  using pre-trained open-weight language models and then fine-tune, in tandem, a regression head and a set of low-rank adaptation (LoRA) blocks (Hu et al., 2022). Further details on the training configuration are available in Appendix B.

**Base Models.** Consistent with previous work (Klein et al., 2025; Xi et al., 2025a;b), we initialize our learned network  $f_\theta$  from the following base LLMs (all of them the instruct variants): Llama-3.2-11B, Llama-3.1-8B (Grattafiori et al., 2024), Qwen-2.5-3B, and Qwen-2.5-7B (Yang et al., 2024). For our prior LLM policy  $\pi$ , we use a broad range of models with varying accessibility levels to demonstrate the generalizability of our approach. Specifically, we evaluated open-weight LLM agents—Llama-3.2-11B and Llama-3.1-8B—and black-box agents—GPT-4.1-mini (OpenAI,

2025a) and GPT-5.1 (OpenAI, 2025b).<sup>4</sup> We once again highlight the compatibility of our method with black-box agents as a key feature for many real-world settings.

**Baselines.** We compare AMC with two primary baselines performing test-time sampling of parallel agent trajectories: Best-of- $N$  and SMC with Fleet-of-Agents (FoA) prompting (Klein et al., 2025). Best-of- $N$  acts as a baseline, selecting the highest-reward sequence from  $N$  parallel trajectories. AMC extends this by using learned value estimates from our optimized value function to guide selection throughout the trajectory, rather than relying solely on terminal outcomes. Our SMC baseline with FoA prompting (Klein et al., 2025), referred to as SMC (FoA), implements trajectory pruning via SMC, leveraging its 2-shot prompt template to evaluate state values based on the preceding trajectory history. Although the original FoA employs tree-based search, we focus on its core prompting logic to provide a direct comparison of state-evaluation quality. Since Klein et al. (2025) only considered WebShop from AgentGym, we compare with their SMC prompting approach on this benchmark. To ensure a fair comparison, we apply SMC (FoA) at the same empirically optimized resampling step used by AMC (Figure 5, Figure 6, and Figure 7 of Appendix D.1). For the agent’s action-generation logic, we mainly adopt a ReAct-style prompting framework (Yao et al., 2023). To examine whether AMC transfers beyond this setting, we also implement a ReAct-style agent (Kim et al., 2025) on SciWorld. We also need to choose the number of trajectories  $N$ . Following the discussion in Section 2.2 in which we note that SMC approximates the optimal policy only as  $N \rightarrow \infty$ , we are compelled to choose a reasonably large (but tractable) trajectory count; to this end, we use  $N = 15$  trajectories throughout.

**Metrics.** In most applications of our method, we can use only a single final trajectory (e.g., in online shopping or deciding on a live chat response), and we will also have access to a (real or approximated) reward function  $r$ . The most practical application of our method is then to choose the highest reward trajectory out of the  $N$  produced. To evaluate this setting, we report performance scores of the highest-reward trajectory per method using the cumulative environmental rewards provided by each benchmark. We conduct three independent trials using different seeds to calculate the mean scores and corresponding standard errors. We also compare to the average scores of the top 5 trajectories within an  $N$ -trajectory pool in Appendix D.5, with AMC uniformly improving average-trajectory performance across tasks. We provide a mathematical proof for this result in Appendix A.4.

<sup>4</sup>We do not use native reasoning capabilities of OpenAI models.

Table 1. WebShop performance, obtained with a Llama-3.2-11B-based value model for AMC.

| Policy Model  | Method             | Score                                 |
|---------------|--------------------|---------------------------------------|
| Llama-3.2-11B | ReAct              | 0.159 ( $\pm 0.030$ )                 |
|               | Best-of-15 (ReAct) | 0.562 ( $\pm 0.012$ )                 |
|               | SMC (FoA-ReAct)    | 0.580 ( $\pm 0.016$ )                 |
|               | AMC (ReAct)        | <b>0.625 (<math>\pm 0.009</math>)</b> |
| GPT-4.1-mini  | ReAct              | 0.113 ( $\pm 0.026$ )                 |
|               | Best-of-15 (ReAct) | 0.403 ( $\pm 0.016$ )                 |
|               | AMC (ReAct)        | <b>0.488 (<math>\pm 0.020</math>)</b> |
| GPT-5.1       | ReAct              | 0.171 ( $\pm 0.017$ )                 |
|               | Best-of-15 (ReAct) | 0.519 ( $\pm 0.013$ )                 |
|               | AMC (ReAct)        | <b>0.543 (<math>\pm 0.009</math>)</b> |

## 4.2. Experimental Results

**WebShop.** We generated  $N = 15$  trajectories for all methods, with the exception of the ReAct baseline, which consists of a single trajectory ( $N = 1$ ). Both AMC and SMC (FoA) used the same Llama-3.2-11B as the base value function for a fair comparison. Table 1 presents the overall performance on WebShop. Scaling the number of sampled trajectories with Best-of-15 consistently yields substantial performance gains over the single-trajectory ReAct baseline. For the Llama-3.2-11B policy, SMC (FoA) provides only marginal improvements over Best-of-15, while AMC clearly outperforms all methods, demonstrating the benefits of learning a value function using AMC compared to the hand-crafted prompt-based value function from SMC (FoA). We also find it noteworthy that our relatively small (11B) value function can successfully guide a large-scale foundational agent like GPT-5.1, improving performance over this base policy without ever updating the underlying model.

**SciWorld.** In SciWorld, we use Llama-3.1-8B as the base model for our learned value functions in order to match the setup of Xi et al. (2025b), who also report results on this dataset. Table 2 presents the overall performance results on the SciWorld benchmark. Similar to our WebShop findings, expanding the trajectory pool with Best-of- $N$  significantly outperforms single-trajectory baselines, while AMC delivers additional consistent gains across all evaluated policies. For this benchmark, we considered both ReAct- and ReAct-style agents; we find that ReAct demonstrates superior performance. Our AMC method with a ReAct-style policy achieved further improvements, highlighting the potential for compounding gains when combining AMC with more advanced decision-making agents. The consistent performance gains observed across all model scales underscore the robustness and scalability of the AMC framework.

**TextCraft.** Lastly, in TextCraft, we generated  $N = 15$  trajectories and trained a Llama-3.2-11B value function for AMC. Table 3 again demonstrates that multiple-trajectory approaches achieve a higher score compared to single-trajectory baselines. AMC also obtained substantial gains over Best-of-15 for smaller policy models. However, we

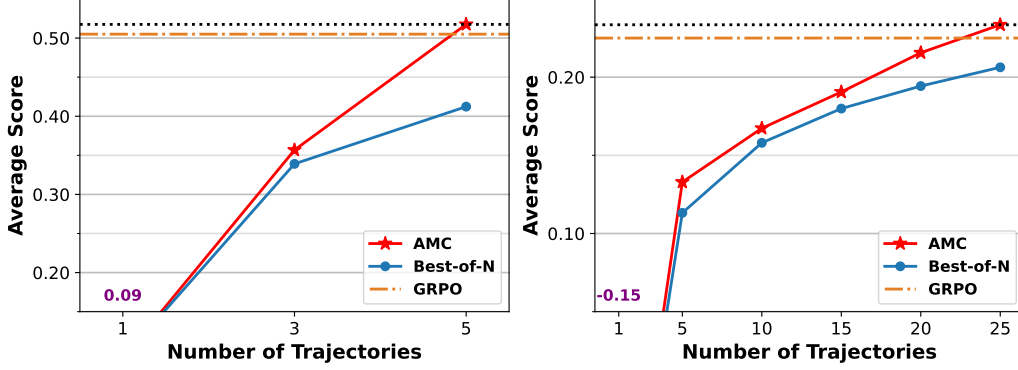


Figure 3. Comparisons to GRPO on SciWorld. Left: AMC and Best-of- $N$  with a GPT-5.1 policy and a Qwen-2.5-7B-based value function, compared to GRPO with a Qwen-2.5-7B backbone (highlighting the advantage of using AMC with black-box models). Right: AMC and Best-of- $N$  with a Qwen-2.5-3B policy and value function, compared to GRPO with the same backbone.

Table 2. SciWorld performance, obtained with a Llama-3.1-8B-based value model for AMC.

| Policy Model | Method               | Score                                 |
|--------------|----------------------|---------------------------------------|
| Llama-3.1-8B | ReAct                | 0.013 ( $\pm 0.014$ )                 |
|              | Best-of-15 (ReAct)   | 0.311 ( $\pm 0.014$ )                 |
|              | AMC (ReAct)          | <b>0.347 (<math>\pm 0.015</math>)</b> |
|              | ReflAct              | 0.051 ( $\pm 0.015$ )                 |
|              | Best-of-15 (ReflAct) | 0.347 ( $\pm 0.010$ )                 |
| GPT-4.1-mini | AMC (ReflAct)        | <b>0.376 (<math>\pm 0.016</math>)</b> |
|              | ReAct                | 0.250 ( $\pm 0.023$ )                 |
|              | Best-of-15 (ReAct)   | 0.616 ( $\pm 0.020$ )                 |
| GPT-5.1      | AMC (ReAct)          | <b>0.673 (<math>\pm 0.009</math>)</b> |
|              | ReAct                | 0.090 ( $\pm 0.049$ )                 |
|              | Best-of-15 (ReAct)   | 0.533 ( $\pm 0.026$ )                 |
|              | AMC (ReAct)          | <b>0.597 (<math>\pm 0.023</math>)</b> |

observe performance saturation as the base policy improves. This is evident with GPT-5.1, where Best-of-15 even exceeds AMC. We hypothesize that this performance inversion is due to the strength of GPT-5.1 as a prior. For TextCraft, GPT-5.1 generates shorter, higher-confidence trajectories, leaving little room for improvement and reducing the diversity of the value function’s training data. This makes it more difficult for AMC to separate out promising trajectories, leading it to accidentally prune some. Based on this finding, we expect AMC to be most useful for model-task pairs where the model produces good but not uniformly perfect trajectories. Crucially, these findings do not imply that AMC loses utility as models get better because (i) there is no foreseeable upper bound on real-world problem complexity and (ii) cost constraints will favour smaller, more efficient models that may not be performance-maximal.

### 4.3. Comparison with GRPO

GRPO (Shao et al., 2024) is a KL-regularized RL method that has become common for LLM-based policies. As with AMC, GRPO attempts to model  $\pi_*$  from Equation 1, but unlike AMC, it requires full access to the policy’s underlying parameters to perform weight updates. GRPO also

Table 3. TextCraft performance, obtained with a Llama-3.2-11B-based value model for AMC.

| Policy Model  | Method             | Score                                 |
|---------------|--------------------|---------------------------------------|
| Llama-3.2-11B | ReAct              | 0.102 ( $\pm 0.029$ )                 |
|               | Best-of-15 (ReAct) | 0.296 ( $\pm 0.019$ )                 |
|               | AMC (ReAct)        | <b>0.543 (<math>\pm 0.057</math>)</b> |
| GPT-4.1-mini  | ReAct              | 0.432 ( $\pm 0.055$ )                 |
|               | Best-of-15 (ReAct) | 0.728 ( $\pm 0.010$ )                 |
|               | AMC (ReAct)        | <b>0.852 (<math>\pm 0.020</math>)</b> |
| GPT-5.1       | ReAct              | 0.691 ( $\pm 0.012$ )                 |
|               | Best-of-15 (ReAct) | <b>0.889 (<math>\pm 0.000</math>)</b> |
|               | AMC (ReAct)        | 0.790 ( $\pm 0.021$ )                 |

requires online trajectory rollouts during training, which incurs additional computational overhead, over the simple offline regression task required to learn  $V_\theta$  (Section 2.3). To clarify, AMC is not intended to replace GRPO, but to serve as a viable alternative when GRPO is not possible. We thus consider GRPO to be an oracle rather than a typical baseline.

In this section, we put AMC and GRPO head-to-head to better understand whether our approach is a viable alternative to policy gradient RL. For a direct comparison to Xi et al. (2025b), we use the same LLM policy backbone (Qwen-2.5-3B) for both AMC and GRPO; we also use this same base model to train our value function, thus controlling for base model and pre-training biases. Additionally, to highlight AMC’s compatibility with black-box policies, we consider an alternative comparison setup: using a frontier LLM agent (GPT-5.1) as the prior policy for AMC and training a value function backed by the same LLM as the one GRPO fine-tunes for its policy (Qwen-2.5-7B).

Figure 3 illustrates these comparisons on the SciWorld benchmark. Note that all GRPO scores are taken from Xi et al. (2025b). When backed by a GPT-5.1 policy (Figure 3, left), AMC outperforms GRPO with only  $N = 5$  trajectories, achieving a performance level that remains unattainable for Best-of- $N$  given the same trajectory budget. Further-

more, when using the same LLM backbone for the policy of both GRPO and AMC (Figure 3, right), we find that when scaling AMC to 25 trajectories it outperforms fully fine-tuning this policy with GRPO. These results highlight that (i) AMC performs comparably to GRPO using the same prior policy given enough trajectories (but requiring fewer than Best-of- $N$ ), and (ii) it can benefit even more from better black-box priors. In Appendix D.4, we fine-tuned our own GRPO baselines to provide a comprehensive performance and cost analysis. To underscore the relative cost requirements of AMC and GRPO, we note that all AMC experiments were performed on a workstation with two RTX 6000 Ada desktop GPUs, whereas GRPO experiments required a node with eight A100 GPUs.

#### 4.4. Impact of Value Model Optimization

**Training vs. Prompting.** Since our trained value functions are initialized from pre-trained (instruction-tuned) LLMs, we ask the question: is training necessary? Can we achieve the same performance by prompting alone? We thus compare AMC to a (heuristic) prompt-based approach, SMC (Zero-shot), where we simply ask the pre-trained LLM to estimate the value of a given state (see Appendix C for all prompts). Table 4 illustrates this comparison across both open-weight and black-box policies, revealing consistent performance gains from training. Notably, SMC (Zero-Shot) delivers inconsistent performance gain over the Best-of- $N$  baseline, underscoring that raw pre-trained knowledge is insufficient for precise state-value estimation in these agentic environments.

#### 4.5. Cost-Efficiency and Scaling

**Trajectory Counts.** The effectiveness of our approach relies on sampling multiple trajectories at test-time, which naturally raises questions around inference cost and scalability. To evaluate the practical feasibility of AMC, we compare its efficiency against Best-of- $N$  in Figure 4. When the number of trajectories is limited to 5 or 10, AMC demonstrates performance gains over the baseline with the exception of TextCraft. This exception is likely due to the reduced diversity of visited states arising from TextCraft’s lower relative task complexity, and is further hampered by AMC’s resampling; increasing the trajectory count resolves this by promoting more state space exploration. As  $N$  increases, AMC generally achieves performance parity with Best-of-20 using only  $N = 15$  trajectories, thereby reducing test-time-compute by approximately 25%. Note that the value function is only evaluated on the initial state and on resampling steps and requires only a single prefill, so its computational overhead is small compared to the cost of the LLM policy, which performs both prefill and decoding at every step.

Table 4. Impact of value function optimization on performance.

| Dataset   | Policy Model  | Method          | Score                                 |
|-----------|---------------|-----------------|---------------------------------------|
| Webshop   | Llama-3.2-11B | Best-of-15      | 0.562 ( $\pm 0.012$ )                 |
|           |               | SMC (Zero-shot) | 0.556 ( $\pm 0.014$ )                 |
|           |               | AMC             | <b>0.625 (<math>\pm 0.009</math>)</b> |
|           | GPT-4.1-mini  | Best-of-15      | 0.403 ( $\pm 0.016$ )                 |
|           |               | SMC (Zero-shot) | 0.418 ( $\pm 0.014$ )                 |
|           |               | AMC             | <b>0.488 (<math>\pm 0.020</math>)</b> |
|           | GPT-5.1       | Best-of-15      | 0.519 ( $\pm 0.013$ )                 |
|           |               | SMC (Zero-shot) | 0.522 ( $\pm 0.008$ )                 |
|           |               | AMC             | <b>0.543 (<math>\pm 0.009</math>)</b> |
| SciWorld  | Llama-3.1-8B  | Best-of-15      | 0.311 ( $\pm 0.014$ )                 |
|           |               | SMC (Zero-shot) | 0.305 ( $\pm 0.013$ )                 |
|           |               | AMC             | <b>0.347 (<math>\pm 0.015</math>)</b> |
|           | GPT-4.1-mini  | Best-of-15      | 0.616 ( $\pm 0.020$ )                 |
|           |               | SMC (Zero-shot) | 0.640 ( $\pm 0.022$ )                 |
|           |               | AMC             | <b>0.673 (<math>\pm 0.009</math>)</b> |
|           | GPT-5.1       | Best-of-15      | 0.533 ( $\pm 0.026$ )                 |
|           |               | SMC (Zero-shot) | 0.506 ( $\pm 0.041$ )                 |
|           |               | AMC             | <b>0.597 (<math>\pm 0.023</math>)</b> |
| TextCraft | Llama-3.2-11B | Best-of-15      | 0.296 ( $\pm 0.019$ )                 |
|           |               | SMC (Zero-shot) | 0.296 ( $\pm 0.043$ )                 |
|           |               | AMC             | <b>0.543 (<math>\pm 0.057</math>)</b> |
|           | GPT-4.1-mini  | Best-of-15      | 0.728 ( $\pm 0.010$ )                 |
|           |               | SMC (Zero-shot) | 0.765 ( $\pm 0.025$ )                 |
|           |               | AMC             | <b>0.852 (<math>\pm 0.020</math>)</b> |
|           | GPT-5.1       | Best-of-15      | <b>0.889 (<math>\pm 0.000</math>)</b> |
|           |               | SMC (Zero-shot) | 0.815 ( $\pm 0.026$ )                 |
|           |               | AMC             | 0.790 ( $\pm 0.021$ )                 |

Table 5. Performance and cost of a lightweight policy using AMC versus the more expensive GPT-5.1.

| Dataset   | Policy Model | Method     | Score                                 | Cost (USD)  |
|-----------|--------------|------------|---------------------------------------|-------------|
| WebShop   | GPT-5.1      | Best-of-15 | <b>0.519 (<math>\pm 0.013</math>)</b> | 0.39        |
|           | GPT-4.1-mini | AMC        | 0.488 ( $\pm 0.020$ )                 | <b>0.14</b> |
| SciWorld  | GPT-5.1      | Best-of-15 | 0.533 ( $\pm 0.026$ )                 | 0.18        |
|           | GPT-4.1-mini | AMC        | <b>0.673 (<math>\pm 0.009</math>)</b> | <b>0.06</b> |
| TextCraft | GPT-5.1      | Best-of-15 | <b>0.889 (<math>\pm 0.000</math>)</b> | 0.45        |
|           | GPT-4.1-mini | AMC        | 0.852 ( $\pm 0.020$ )                 | <b>0.21</b> |

**Policy Cost.** Expanding on these results, we find that our method also enables smaller black-box models to achieve performance comparable to their larger counterparts, effectively narrowing the gap between lightweight and high-capacity policies. We empirically substantiate this finding in Table 5, where costs are calculated based on the average input and output token counts across tasks.<sup>5</sup> GPT-4.1-mini paired with AMC maintains a performance level similar to GPT-5.1 while reducing total cost by at least 50%.

**Prior Surrogates.** Part of AMC’s upfront cost is the initial collection of trajectories from the prior LLM policy  $\pi$  to train the value model. When  $\pi$  is behind an API, this upfront data acquisition can become expensive. To circumvent this, we explore using small, open-weight surrogate policies to sample training trajectories instead of using  $\pi$  directly; lightweight models can generally be self-hosted on existing GPUs to circumvent API costs. Table 6 shows results; using a surrogate policy uniformly outperforms Best-of- $N$ . Although sampling trajectories from  $\pi$  itself performs better

<sup>5</sup>OpenAI pricing page.

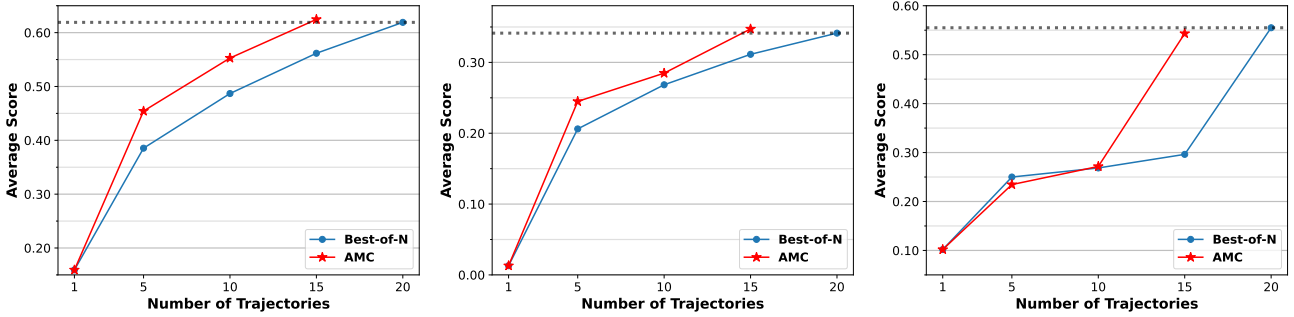


Figure 4. Best-of- $N$  vs. AMC across tasks. Left: WebShop with Llama-3.2-11B policy, value function. Middle: SciWorld with Llama-3.1-8B policy, value function. Right: TextCraft with Llama-3.2-11B policy, value function.

Table 6. Transferability of value functions where base models are Llama-3.2-11B (WebShop, TextCraft) and Llama-3.1-8B (SciWorld).

| Dataset   | Policy Model | Method     | Surrogate     | Score                                 |
|-----------|--------------|------------|---------------|---------------------------------------|
| WebShop   | GPT-4.1-mini | Best-of-15 | -             | 0.403 ( $\pm 0.016$ )                 |
|           |              | AMC        | Llama-3.2-11B | <b>0.467 (<math>\pm 0.028</math>)</b> |
|           |              |            | None          | <b>0.488 (<math>\pm 0.020</math>)</b> |
| SciWorld  | GPT-4.1-mini | Best-of-15 | -             | 0.616 ( $\pm 0.020$ )                 |
|           |              | AMC        | Llama-3.1-8B  | <b>0.645 (<math>\pm 0.023</math>)</b> |
|           |              |            | None          | <b>0.673 (<math>\pm 0.009</math>)</b> |
| TextCraft | GPT-4.1-mini | Best-of-15 | -             | 0.728 ( $\pm 0.010$ )                 |
|           |              | AMC        | Llama-3.2-11B | <b>0.790 (<math>\pm 0.025</math>)</b> |
|           |              |            | None          | <b>0.852 (<math>\pm 0.020</math>)</b> |

than using a surrogate policy, these results still demonstrate transferability of  $V_\theta$  as a viable way to reduce cost.

## 5. Conclusion, Limitations, and Future Work

Most of today’s LLM-based interactions are hidden behind black-box APIs, and yet most agentic applications (e.g., e-commerce, software engineering) are specialized tasks that could benefit from task-specific tuning. Is it possible to emulate the effects of RL-based fine-tuning with black-box policies? Our work answers this question in the affirmative. Following the well-known bridge between KL-regularized RL and Bayesian inference, we approximate the agent’s optimal policy without directly training it through RL, but rather by Monte Carlo sampling from its Bayesian posterior distribution. Our algorithm, Agentic Monte Carlo (AMC), implements sequential importance resampling using a learned value function to guide samples from the black-box LLM prior toward samples from the intractable optimal posterior.

We empirically showed the effectiveness of our approach on the WebShop, SciWorld, and TextCraft benchmarks, highlighting improvements over existing baselines. Notably, AMC is able to improve the performance of smaller black-box models to match that of larger black-box models, thus incurring significantly cheaper API costs. We further showcased our method against policy gradient RL methods such as GRPO, demonstrating that AMC can achieve equivalent performance as we scale test-time compute.

There are a number of avenues for future work on AMC. For one, our learned value function will never perfectly approximate the true value function. Potential ways to improve the training procedure include temporal difference learning, (Sutton, 1988), reward shaping (Ng et al., 1999), or better approximating Equation 5. Another area for improvement is the computational overhead that comes with having to sample multiple trajectories in parallel during inference. Greater efficiency may be attained, for example, by distilling AMC or some component of AMC into a smaller network (Hinton et al., 2015). AMC can also be extended to broader settings, like multi-agent black-box agents. As more frontier models are released with proprietary, opaque architectures, AMC provides a principled mechanism to orient and optimize these black-box agents for task-specific execution. Additionally, several promising directions exist for enhancing resampling strategies: developing auxiliary models trained to trigger resampling, leveraging LLM-based judges to identify specific trajectory states where resampling is likely to yield the highest marginal gain (Feng et al., 2026), and employing adaptive resampling frameworks or hyperparameter optimization techniques to arrive at more sophisticated choices of resampling intervals (Doucet et al., 2001; Chopin et al., 2020). Lastly, there remains theory to be built around our method. AMC introduces potential error in the reward model (Huang et al., 2025) or value function, and it would be useful to understand its interplay with the already-extensive body of work around SMC (Crisan & Doucet, 2000; Moral, 2004; Marion et al., 2023).

## Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning, specifically the use of LLM agents. Our work focuses on providing statistically principled sampling methods to extend reinforcement learning techniques to black-box LLMs. This work is unlikely to cause broader societal impact outside of the general impacts of agentic AI.

## References

- Anthropic. Introducing Claude Opus 4.6, 2026. URL <https://www.anthropic.com/news/claude-opus-4-6>.
- Chopin, N., Papaspiliopoulos, O., et al. *An introduction to sequential Monte Carlo*, volume 4. Springer, 2020.
- Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Crisan, D. and Doucet, A. Convergence of sequential monte carlo methods. *Signal Processing Group, Department of Engineering, University of Cambridge, Technical Report CUEDIF-INFENGrrR38*, 1(525):92, 2000.
- Dayan, P. and Hinton, G. E. Using expectation-maximization for reinforcement learning. *Neural Computation*, 9(2):271–278, 1997.
- Doucet, A., Godsill, S., and Andrieu, C. On sequential Monte Carlo sampling methods for Bayesian filtering. *Statistics and Computing*, 10(3):197–208, 2000. ISSN 1573-1375. doi: 10.1023/A:1008935410038.
- Doucet, A., de Freitas, N., and Gordon, N. *An Introduction to Sequential Monte Carlo Methods*, pp. 3–14. Springer New York, 2001. ISBN 978-1-4757-3437-9. doi: 10.1007/978-1-4757-3437-9.1.
- Feng, N., Sui, Y., Hou, S., Wu, G., and Cresswell, J. C. Conformal Agent Error Attribution. *arXiv:2605.06788*, 2026.
- Google. Gemini 3 Pro Model Card, November 2025. URL <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf>.
- Gordon, N. J., Salmond, D. J., and Smith, A. F. Novel approach to nonlinear/non-gaussian bayesian state estimation. In *IEE proceedings F (Radar and Signal Processing)*, volume 140, pp. 107–113, 1993.
- Grattafiori, A. et al. The Llama 3 Herd of Models. *arXiv:2407.21783*, 2024.
- Haarnoja, T., Tang, H., Abbeel, P., and Levine, S. Reinforcement learning with deep energy-based policies. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 1352–1361, 2017.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pp. 1861–1870, 2018.
- Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.
- Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Huang, A., Block, A., Liu, Q., Jiang, N., Krishnamurthy, A., and Foster, D. J. Is best-of-n the best of them? coverage, scaling, and optimality in inference-time alignment. In *International Conference on Machine Learning*, pp. 25075–25126. PMLR, 2025.
- Kim, J., Rhee, S., Kim, M., Kim, D., Lee, S., Sung, Y., and Jung, K. ReAct: World-Grounded Decision Making in LLM Agents via Goal-State Reflection. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1697.
- Klein, L. H., Potamitis, N., Aydin, R., West, R., Gulcehre, C., and Arora, A. Fleet of agents: Coordinated problem solving with large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 30986–31019, 2025.
- Korbak, T., Perez, E., and Buckley, C. RL with KL penalties is better viewed as Bayesian inference. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 1083–1091. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.findings-emnlp.77.
- Levine, S. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv:1805.00909*, 2018.
- Lew, A. K., Zhi-Xuan, T., Grand, G., and Mansinghka, V. Sequential Monte Carlo Steering of Large Language Models using Probabilistic Programs. In *ICML 2023 Workshop: Sampling and Optimization in Discrete Space*, 2023.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s Verify Step by Step. In *The Twelfth International Conference on Learning Representations*, 2024.
- Lioutas, V., Lavington, J. W., Sefas, J., Niedoba, M., Liu, Y., Zwartsenberg, B., Dabiri, S., Wood, F., and Scibior, A. Critic Sequential Monte Carlo. In *The Eleventh International Conference on Learning Representations*, 2023.
- Loula, J., LeBrun, B., Du, L., Lipkin, B., Pasti, C., Grand, G., Liu, T., Emara, Y., Freedman, M., Eisner, J., Cotterell, R., Mansinghka, V., Lew, A. K., Vieira, T., and

- O'Donnell, T. J. Syntactic and Semantic Control of Large Language Models via Sequential Monte Carlo. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Ma, C., Zhang, J., Zhu, Z., Yang, C., Yang, Y., Jin, Y., Lan, Z., Kong, L., and He, J. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, pp. 74325–74362, 2024. doi: 10.52202/079017-2365.
- Marion, J., Mathews, J., and Schmidler, S. C. Finite-sample complexity of sequential monte carlo estimators. *The Annals of Statistics*, 51(3):1357–1375, 2023.
- Moral, P. *Feynman-Kac formulae: genealogical and interacting particle systems with applications*. Springer, 2004.
- Ng, A. Y., Harada, D., and Russell, S. J. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pp. 278–287. Morgan Kaufmann Publishers Inc., 1999. ISBN 1558606122.
- OpenAI. Introducing GPT-4.1 in the API. <https://openai.com/index/gpt-4-1/>, 2025a. Released April 14, 2025.
- OpenAI. GPT-5 System Card, August 2025b. URL <https://cdn.openai.com/gpt-5-system-card.pdf>.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pp. 27730–27744, 2022.
- Pennino, F., Raimondi, B., Rondelli, M., Gurioli, A., and Gabbrielli, M. From Reasoning to Code: GRPO Optimization for Underrepresented Languages. *arXiv:2506.11027*, 2025.
- Piché, A., Thomas, V., Ibrahim, C., Bengio, Y., and Pal, C. Probabilistic Planning with Sequential Monte Carlo methods. In *International Conference on Learning Representations*, 2019.
- Prasad, A., Koller, A., Hartmann, M., Clark, P., Sabharwal, A., Bansal, M., and Khot, T. ADaPT: As-Needed Decomposition and Planning with Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 4226–4252, 2024. doi: 10.18653/v1/2024.findings-naacl.264.
- Puri, I., Sudalairaj, S., Xu, G., Bhandwaladar, A., Xu, K., and Srivastava, A. Rollout Roulette: A Probabilistic Inference Approach to Inference-Time Scaling of LLMs using Particle-Based Monte Carlo Methods. In *Advances in Neural Information Processing Systems*, volume 38, pp. 156427–156454, 2025.
- Ross, B. L., Vouitsis, N., Ghomi, A. A., Hosseinzadeh, R., Xin, J., Liu, Z., Sui, Y., Hou, S., Leung, K. K., Loaiza-Ganem, G., and Cresswell, J. C. Textual Bayes: Quantifying Prompt Uncertainty in LLM-Based Systems. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, pp. 68539–68551, 2023.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeek-Math: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv:1712.01815*, 2017.
- Sutton, R. S. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. ISSN 1573-0565. doi: 10.1007/BF00115009.
- Sutton, R. S., Barto, A. G., et al. *Reinforcement learning: An introduction*, volume 1. 1998.
- Toussaint, M. and Storkey, A. Probabilistic inference for solving discrete and continuous state Markov Decision Processes. In *Proceedings of the 23rd International Conference on Machine Learning*, pp. 945–952. Association for Computing Machinery, 2006. ISBN 1595933832. doi: 10.1145/1143844.1143963.
- Wang, R., Jansen, P., Côté, M.-A., and Ammanabrolu, P. ScienceWorld: Is your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11279–11298, 2022. doi: 10.18653/v1/2022.emnlp-main.775.

- Wang, Y., Yang, Q., Zeng, Z., Ren, L., Liu, L., Peng, B., Cheng, H., He, X., Wang, K., Gao, J., Chen, W., Wang, S., Du, S., and shen, y. Reinforcement Learning for Reasoning in Large Language Models with One Training Example. In *Advances in Neural Information Processing Systems*, volume 38, pp. 122721–122764, 2025.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.
- Xi, Z., Ding, Y., Chen, W., Hong, B., Guo, H., Wang, J., Guo, X., Yang, D., Liao, C., He, W., Gao, S., Chen, L., Zheng, R., Zou, Y., Gui, T., Zhang, Q., Qiu, X., Huang, X., Wu, Z., and Jiang, Y.-G. AgentGym: Evaluating and training large language model-based agents across diverse environments. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27914–27961, 2025a. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1355.
- Xi, Z., Huang, J., Liao, C., Huang, B., Guo, H., Liu, J., Zheng, R., Ye, J., Zhang, J., Chen, W., He, W., Ding, Y., Li, G., Chen, Z., Du, Z., Yao, X., Xu, Y., Chen, J., Gui, T., Wu, Z., Zhang, Q., Huang, X., and Jiang, Y.-G. AgentGym-RL: Training LLM Agents for Long-Horizon Decision Making through Multi-Turn Reinforcement Learning. *arXiv:2509.08755*, 2025b.
- Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 Technical Report. *arXiv:2412.15115*, 2024.
- Yao, S., Chen, H., Yang, J., and Narasimhan, K. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, volume 35, pp. 20744–20757, 2022.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*, 2023.
- Yu, X., Peng, B., Vajipey, V., Cheng, H., Galley, M., Gao, J., and Yu, Z. ExACT: Teaching AI Agents to Explore with Reflective-MCTS and Exploratory Learning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Zhao, S., Brekelmans, R., Makhzani, A., and Grosse, R. B. Probabilistic inference in language models via twisted sequential Monte Carlo. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 60704–60748, 2024.
- Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., and Wang, Y.-X. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 62138–62160, 2024.
- Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., Christiano, P., and Irving, G. Fine-tuning language models from human preferences. *arXiv:1909.08593*, 2019.

## A. Theory

### A.1. Sequential Importance Resampling

In this section, we record the full SIR algorithm, [Algorithm 1](#) (Doucet et al., 2001), with our weight update rule including the (soft) value function.

---

#### Algorithm 1 Sequential Importance Resampling

---

**Require:** Reward function  $r$ , value function  $V$ , prior policy  $\pi$ , resampling criterion  $c$ , trajectory count  $N$ , time horizon  $T$

**Ensure:** Posterior trajectories  $s_{0:T}^{(i)}$  and weights  $w_T^{(i)}$  for  $i = 1, \dots, N$

- 1: **Initialization:** For  $i = 1, \dots, N$ , sample  $s_0^{(i)} \sim \pi(s_0)$  and set  $w_0^{(i)} = 1/N$ .
  - 2: **for**  $t = 1$  **to**  $T$  **do**
  - 3:   **for**  $i = 1$  **to**  $N$  **do**
  - 4:     Sample  $\tilde{s}_t^{(i)} \sim \pi(s_t \mid s_{t-1}^{(i)})$
  - 5:     Set  $\tilde{s}_{0:t}^{(i)} \leftarrow (s_{0:t-1}^{(i)}, \tilde{s}_t^{(i)})$
  - 6:     Compute importance weights:
 
$$w_t^{(i)} = w_{t-1}^{(i)} e^{V(\tilde{s}_t^{(i)}) - V(s_{t-1}^{(i)}) + r(s_{t-1}^{(i)})}$$
  - 7:   **if**  $c(t, \{w_t^{(i)}\}_{i=1}^N)$  **then**
  - 8:     Resample  $N$  choices of trajectory  $\{s_{0:t}^{(i)}\}_{i=1}^N$  from  $\{\tilde{s}_{0:t}^{(i)}\}_{i=1}^N$  according to the normalized weights:
 
$$\text{Multinomial}\left(N, \left\{\frac{w_t^{(i)}}{\sum_{j=1}^N w_t^{(j)}}\right\}_{i=1}^N\right).$$
  - 9:     Reset weights  $w_t^{(i)} \leftarrow 1/N$ .
  - 10:   **else**
  - 11:      $s_{0:t}^{(i)} \leftarrow \tilde{s}_{0:t}^{(i)}$  for  $i = 1, \dots, N$ .
- 

### A.2. Equivalence between KL-Regularized RL and Bayesian Inference

For completeness, we include the proof of the equivalence between KL-regularized RL and Bayesian inference discussed by [Korbak et al. \(2022\)](#). This is also a known result in the control-as-inference literature ([Levine, 2018](#)), and mathematically the same as the fundamental proof justifying variational inference (VI) as a method.

**Proposition A.1.** *If we let*

$$\pi_*^{(\text{Bayes})}(s_{0:T}) = \frac{1}{Z} \pi(s_{0:T}) e^{r(s_{0:T})/\beta}, \quad (8)$$

where  $Z = \int \pi(s_{0:T}) e^{r(s_{0:T})/\beta} ds_{0:T}$  is a normalizing constant, and let

$$\pi_*^{(\text{VI})} = \arg \max_{\pi_\theta} \mathcal{J}(\pi_\theta) = \arg \max_{\pi_\theta} \left( \mathbb{E}_{\pi_\theta(s_{0:T})} [r(s_{0:T})] - \beta \text{KL}[\pi_\theta \parallel \pi] \right), \quad (9)$$

then  $\pi_*^{(\text{Bayes})} = \pi_*^{(\text{VI})}$ .

*Proof.* Expanding  $\mathcal{J}(\pi_\theta)$ , we can rewrite it in terms of  $\pi_*^{(\text{Bayes})}$ :

$$\begin{aligned}
 \mathcal{J}(\pi_\theta) &= \mathbb{E}_{\pi_\theta(s_{0:T})} [r(s_{0:T})] - \beta \mathbb{KL} [\pi_\theta \| \pi] \\
 &= \int \pi_\theta(s_{0:T}) r(s_{0:T}) ds_{0:T} - \beta \int \pi_\theta(s_{0:T}) \log \frac{\pi_\theta(s_{0:T})}{\pi(s_{0:T})} ds_{0:T} \\
 &= \beta \int \pi_\theta(s_{0:T}) \left[ \frac{r(s_{0:T})}{\beta} - \log \frac{\pi_\theta(s_{0:T})}{\pi(s_{0:T})} \right] ds_{0:T} \\
 &= \beta \int \pi_\theta(s_{0:T}) \log \frac{\pi(s_{0:T}) e^{r(s_{0:T})/\beta}}{\pi_\theta(s_{0:T})} ds_{0:T} \\
 &= \beta \int \pi_\theta(s_{0:T}) \log \frac{Z \pi_*^{(\text{Bayes})}(s_{0:T})}{\pi_\theta(s_{0:T})} ds_{0:T} \\
 &= \beta \log Z - \beta \int \pi_\theta(s_{0:T}) \log \frac{\pi_\theta(s_{0:T})}{\pi_*^{(\text{Bayes})}(s_{0:T})} ds_{0:T} \\
 &= \beta \log Z - \beta \mathbb{KL} [\pi_\theta \| \pi_*^{(\text{Bayes})}]
 \end{aligned}$$

Because  $\beta \log Z$  is a constant independent of  $\pi_\theta$ , the argmax of  $\mathcal{J}(\pi_\theta)$  is equivalent to the argmin of  $\mathbb{KL} [\pi_\theta \| \pi_*^{(\text{Bayes})}]$ . Since the KL divergence is minimized uniquely at zero when its arguments are equal, it follows that  $\pi_*^{(\text{VI})} = \pi_*^{(\text{Bayes})}$ .  $\square$

### A.3. Recursive Decomposition of Importance Weights

Here we show how the importance weights can be recursively decomposed. This proof is similar, for example, to that of [Piché et al. \(2019, Appendix A.4\)](#).

**Proposition A.2.** *The importance weight  $w_t = \frac{\pi_*(s_{0:t})}{\pi(s_{0:t})}$  (Equation 3) obeys the recursive decomposition from Equation 4:*

$$w_t = w_{t-1} \cdot e^{V(s_t) - V(s_{t-1}) + \frac{r(s_{t-1})}{\beta}},$$

where  $V(s_t) = \log \mathbb{E}_{\pi(s_{t+1:T}|s_t)} [e^{\frac{1}{\beta} \sum_{\tau=t}^T r(s_\tau)}]$  is the soft value function.

*Proof.* First, we expand the marginal posterior  $\pi_*(s_{0:t})$  using the definition of  $\pi_*$  from Equation 2:

$$\begin{aligned}
 \pi_*(s_{0:t}) &= \int \pi_*(s_{0:T}) ds_{t+1:T} \\
 &= \frac{1}{Z} \int \pi(s_{0:T}) e^{\frac{1}{\beta} \sum_{\tau=0}^T r(s_\tau)} ds_{t+1:T} \\
 &= \frac{1}{Z} \pi(s_{0:t}) e^{\frac{1}{\beta} \sum_{\tau=0}^{t-1} r(s_\tau)} \int \pi(s_{t+1:T} | s_t) e^{\frac{1}{\beta} \sum_{\tau=t}^T r(s_\tau)} ds_{t+1:T} \\
 &= \frac{1}{Z} \pi(s_{0:t}) e^{\frac{1}{\beta} \sum_{\tau=0}^{t-1} r(s_\tau)} e^{V(s_t)}.
 \end{aligned}$$

We then use this expression to expand out the importance weight:

$$\begin{aligned}
 w_t &= \frac{\pi_*(s_{0:t})}{\pi(s_{0:t})} \\
 &= \frac{1}{Z} e^{\frac{1}{\beta} \sum_{\tau=0}^{t-1} r(s_\tau)} e^{V(s_t)}.
 \end{aligned}$$

From this expression, we can isolate  $w_{t-1}$  to get a recursive decomposition:

$$\begin{aligned}
 w_t &= \left( \frac{1}{Z} e^{\frac{1}{\beta} \sum_{\tau=0}^{t-2} r(s_\tau)} e^{V(s_{t-1})} \right) \cdot \frac{e^{\frac{1}{\beta} r(s_{t-1})} e^{V(s_t)}}{e^{V(s_{t-1})}} \\
 &= w_{t-1} \cdot e^{V(s_t) - V(s_{t-1}) + \frac{r(s_{t-1})}{\beta}}.
 \end{aligned}$$

$\square$

#### A.4. SMC Improves the Average-Case Trajectory

Here, we show in a simplified setting that SMC will always improve the expected average reward across  $N$  trajectories. In a setting with sparse, binary rewards, we can express the usefulness of a state  $s_t^{(i)}$  by its success probability,  $p_t^{(i)} = P(r(s_T^{(i)}) = 1 \mid s_t^{(i)})$ .

- Given  $N$  different states  $\{s_t^{(1)}, \dots, s_t^{(N)}\}$  and any step  $t$ , the expected average reward over the  $N$  trajectories in a single Best-of- $N$  run is:

$$\mathbb{E}[\text{RewardN}] = \frac{1}{N} \sum_{i=1}^N p_t^{(i)}$$

- On the other hand, with AMC (assuming a perfectly learned value function) let us assume resampling occurs once at timestep  $t$ . Here, we have a simplified value function and weight, so that:

- $V(s_t^{(i)}) = \log[ep_t^{(i)} + (1 - p_t^{(i)})]$ . This comes from the definition of  $V(\cdot)$  (under Equation 4), setting  $r(s_t) = 0$  for  $t \neq T$ , and from expanding out the expectation in the definition with binary rewards. (For this proof we set  $\beta = 1$ .)
- The unnormalized weights are:

$$w_t^{(i)} = e^{V(s_t^{(i)}) - V(s_0^{(i)})} = \frac{ep_t^{(i)} + 1 - p_t^{(i)}}{ep_0^{(i)} + 1 - p_0^{(i)}} = \frac{(e-1)p_t^{(i)} + 1}{(e-1)p_0^{(i)} + 1}$$

- We also let  $\bar{w}_t^{(i)}$  for  $i = 1, \dots, N$  be the renormalized weights used for sampling at timestep  $t$ . Since all trajectories share the same initial state, the denominator cancels out during normalization, yielding the following value:

$$\bar{w}_t^{(i)} = \frac{(e-1)p_t^{(i)} + 1}{(e-1) \sum_{j=1}^N p_t^{(j)} + N}$$

The average resulting reward will be:

$$\begin{aligned} \mathbb{E}[\text{RewardAMC}] &= \frac{1}{N} \sum_{k=1}^N \mathbb{E}_{\tilde{s}_t^{(k)} \sim \bar{w}_t} [\tilde{p}_t^{(k)}] \\ &= \frac{1}{N} \sum_{k=1}^N \left( \sum_{i=1}^N \bar{w}_t^{(i)} p_t^{(i)} \right) \\ &= \sum_{i=1}^N \bar{w}_t^{(i)} p_t^{(i)} \\ &= \sum_{i=1}^N \left( \frac{(e-1)p_t^{(i)} + 1}{(e-1) \sum_{j=1}^N p_t^{(j)} + N} \right) p_t^{(i)} \\ &= \frac{(e-1) \sum_{i=1}^N (p_t^{(i)})^2 + \sum_{i=1}^N p_t^{(i)}}{(e-1) \sum_{j=1}^N p_t^{(j)} + N} \\ &\geq \frac{(e-1) \left[ \frac{1}{N} \left( \sum_{i=1}^N p_t^{(i)} \right)^2 \right] + \sum_{i=1}^N p_t^{(i)}}{(e-1) \sum_{j=1}^N p_t^{(j)} + N} \quad (\text{Cauchy-Schwarz inequality}) \\ &= \frac{\frac{1}{N} \left( \sum_{i=1}^N p_t^{(i)} \right) \left[ (e-1) \sum_{i=1}^N p_t^{(i)} + N \right]}{(e-1) \sum_{j=1}^N p_t^{(j)} + N} \\ &= \frac{1}{N} \sum_{i=1}^N p_t^{(i)} \\ &= \mathbb{E}[\text{RewardN}] \end{aligned}$$

where the **inequality is strict** unless all states have the same success probability (which is essentially never true in practice). Hence, AMC will nearly always result in higher average reward compared to the average of the  $N$  trajectories in Best-of- $N$ . We note that in this theoretical setup, AMC will not always have higher maximum reward, particularly when there already exist very strong trajectories (e.g., a two-trajectory binary success case where one trajectory is guaranteed to succeed, and one is guaranteed to fail).

## B. Experiment Setup

**Hardware and Computational Costs.** All AMC experiments were conducted on a workstation equipped with two NVIDIA RTX 6000 Ada Generation GPUs (48GB VRAM each) and 1TB of system RAM. Under this configuration, the training duration for the Llama-3.1-8B value model was approximately 2 hours, while the Llama-3.2-11B value model required roughly 3 hours to complete. A comprehensive specification of hyperparameters is detailed in Table 7.

**Experimental Protocol.** To ensure statistical robustness and account for stochasticity in model generation, every experiment was executed across three independent trials using distinct random seeds. We report the mean scores alongside their corresponding standard errors to provide a clear measure of performance stability and variance.

Table 7. Detailed implementation settings.

| Category                 | Hyperparameter          | Value                                               |
|--------------------------|-------------------------|-----------------------------------------------------|
| Value Model Optimization | Optimizer               | AdamW                                               |
|                          | Learning rate           | $1 \times 10^{-5}$                                  |
|                          | Scheduler               | Cosine                                              |
|                          | Warmup ratio            | $1 \times 10^{-1}$                                  |
|                          | Max gradient norm       | 1                                                   |
|                          | Loss function           | Exponentially weighted MSE                          |
|                          | Training epochs         | 3                                                   |
|                          | LoRA rank ( $r$ )       | 8                                                   |
|                          | LoRA alpha ( $\alpha$ ) | 16                                                  |
|                          | Output format           | Scalar (float) value                                |
| Inference Configuration  | Temperature ( $\beta$ ) | 1.0                                                 |
|                          | Top- $p$                | 0.95                                                |
|                          | Max output length       | 4096                                                |
|                          | Max steps               | WebShop: 10<br>SciWorld: 20<br>TextCraft: 20        |
|                          |                         | WebShop: [6]<br>SciWorld: [4, 12]<br>TextCraft: [4] |
|                          | Resampling steps        |                                                     |

**Datasets.** We evaluate our framework on three distinct environments. **WebShop** is a simulated e-commerce environment that evaluates an agent’s ability to navigate a multi-step web interface to purchase products satisfying multi-attribute constraints. It features an open-ended action space and a continuous reward based on attribute overlap with the target item. **SciWorld** evaluates long-horizon reasoning through elementary science tasks within a text-based simulator. This environment features a massive combinatorial action space of approximately 200K action-object combinations per step. Success requires strict adherence to procedural instructions, as deviation from the required sequence can trigger a terminal failure reward of -1. Performance is measured by sub-goal milestones, where agents earn incremental points for completing discrete required stages of an experiment. **TextCraft** is a Minecraft-inspired benchmark where agents craft items using hierarchical recipes. Using three primary actions (get, craft, and inventory), agents must manage compositional dependencies to earn a sparse binary reward of 1, granted only upon successful creation of the target item. While the environment features recipes across four levels of depth, we evaluated tasks at depths 3 and 4 since performance on lower-depth tasks has already reached saturation with current policy models (Xi et al., 2025b).

**Training Set Construction.** We first sampled  $\mathcal{G}$  trajectories per task and only included in our dataset the sample with the

highest cumulative reward. In total, we generated  $M = 1.4\text{k}$  trajectories for WebShop and  $1.1\text{k}$  for SciWorld, both using  $\mathcal{G} = 3$ . For TextCraft, given the limited availability of only 44 tasks, we set  $\mathcal{G} = 8$  to promote more diversity in the sampled trajectories. Following previous works (Xi et al., 2025a;b), the maximum number of steps was fixed to  $T = 10$  for WebShop and  $T = 20$  for both SciWorld and TextCraft. By treating each individual step within a trajectory as a separate data point (see Section 2.3), we produced approximately  $P = 14\text{k}$ ,  $22\text{k}$ , and  $880$  samples for WebShop, SciWorld, and TextCraft, respectively, which we then further split 80%/20% for our training and validation sets.

## C. Prompt Templates

### ReAct Prompt Template for WebShop

```

You are web shopping.
I will give you instructions about what to do.
You have to follow the instructions.
Every round I will give you an observation and a list of available actions, you have
to respond an action based on the state and instruction.
You can use search action if search is available.
You can click one of the buttons in clickables.
An action should be of the following structure:
search[keywords]
click[value]
If the action is not valid, perform nothing.
Keywords in search are up to you, but the value in click must be a value in the list
of available actions.
Remember that your keywords in search should be carefully designed.
Your response should use the following format:

Thought:
I think ...

Action:
click[something]
```

## ReAct Prompt Template for SciWorld

You are an agent for science world. Every round I will give you an observation, you have to respond an action based on the observation to finish the given task. Here are the actions you may take:

```
[{"action": "open/close OBJ", "description": "open/close a container",
"action": "de/activate OBJ", "description": "activate/deactivate a device",
"action": "connect OBJ to OBJ", "description": "connect electrical components",
"action": "disconnect OBJ", "description": "disconnect electrical components",
"action": "use OBJ [on OBJ]", "description": "use a device/item",
"action": "look around", "description": "describe the current room",
"action": "look at OBJ", "description": "describe an object in detail",
"action": "look in OBJ", "description": "describe a container's contents",
"action": "read OBJ", "description": "read a note or book",
"action": "move OBJ to OBJ", "description": "move an object to a container",
"action": "pick up OBJ", "description": "move an object to the inventory",
"action": "put down OBJ", "description": "drop an inventory item",
"action": "pour OBJ into OBJ", "description": "pour a liquid into a container",
"action": "dunk OBJ into OBJ", "description": "dunk a container into a liquid",
"action": "mix OBJ", "description": "chemically mix a container",
"action": "go to LOC", "description": "move to a new location",
"action": "eat OBJ", "description": "eat a food",
"action": "flush OBJ", "description": "flush a toilet",
"action": "focus on OBJ", "description": "signal intent on a task object",
"action": "wait", "description": "take no action for 10 iterations",
"action": "wait1", "description": "take no action for 1 iteration",
"action": "examine OBJ", "description": "provides a description of the objects present
on or in a receptacle.",
"action": "task", "description": "describe current task",
"action": "inventory", "description": "list your inventory"]
```

Your response should use the following format:

Thought:

your thoughts.

Action:

your next action

### ReAct Prompt Template for TextCraft

You are given few useful crafting recipes to craft items in Minecraft. Crafting commands are of the format "craft [target object] using [input ingredients]". Every round I will give you an observation, you have to respond an action based on the state and instruction. You can "get" an object (ingredients) from the inventory or the environment, look-up the game inventory by "inventory", or "craft" (target) using any of the crafting commands. Your output must strictly follow this format:

"Thought:  
your thoughts.

Action:  
your next action"

Reminder:

1. Always specify the quantity when using "get" and "craft" commands.
  - Example of get: get 1 lapis lazuli
  - Example1 of craft: craft 1 blue dye using 1 lapis lazuli
  - Example2 of craft: craft 1 golden carrot using 8 gold nugget, 1 carrot
2. When using "get" command, do not specify whether the item comes from the inventory or the environment.
3. You can use ONLY crafting commands provided, do not use your own crafting commands. However, if the crafting command uses a generic ingredient like "planks", you can use special types of the same ingredient e.g. "dark oak planks" in the command instead.

### Value Function Prompt Template for WebShop

You are a value estimator for a web shopping task. Your job is to estimate the expected future reward (return) from the current STATE, given the INSTRUCTION and the previous STATES, ACTIONS provided as context.

Format of the input you will receive:

- INSTRUCTION: The original shopping request to evaluate progress against.
- HISTORY: Consecutive pairs of (STATE, ACTION) taken so far.
- NOW: Current STATE.

Use the INSTRUCTION with HISTORY and NOW to infer progress and likelihood of success.

Focus on attributes, type, options, and price of shown products relative to the INSTRUCTION.

Based on your analysis, provide a score between 0.0 and 1.0.

=== INSTRUCTION ===  
{Instruction}

=== HISTORY ===  
{History}

=== NOW ===  
{Current state}

### Value Function Prompt Template for SciWorld

You are a value estimator for a science world task.  
Your job is to estimate the expected future reward (return) from the current STATE, given the INSTRUCTION and the previous STATES, ACTIONS provided as context.

Format of the input you will receive:

- INSTRUCTION: The original scientific request to evaluate progress against.
- HISTORY: Consecutive pairs of (STATE, ACTION) taken so far.
- NOW: Current STATE.

Use the INSTRUCTION with HISTORY and NOW to infer progress and likelihood of success.

Focus on milestones, such as locating the correct room, acquiring required tools, and following the specific steps (e.g., 'focus', 'interaction') outlined in the INSTRUCTION.

Heavily penalize states where the observation is 'No known action matches that input', as this indicates the agent is stuck in an invalid command loop or syntax error.

Based on your analysis, provide a score between -1.0 and 1.0.

```
=== INSTRUCTION ===
{Instruction}
```

```
=== HISTORY ===
{History}
```

```
=== NOW ===
{Current state}
```

### Value Function Prompt Template for TextCraft

You are a value estimator for a Minecraft crafting task.  
Your job is to estimate the expected future reward (return) from the current STATE, given the INSTRUCTION and the previous STATES, ACTIONS provided as context.

Format of the input you will receive:

- INSTRUCTION: The original crafting request to evaluate progress against.
- HISTORY: Consecutive pairs of (STATE, ACTION) taken so far.
- NOW: Current STATE.

Use the INSTRUCTION with HISTORY and NOW to infer progress and likelihood of success.

Focus on the crafting material, intermediate crafting results and quantities relative to the INSTRUCTION.

Based on your analysis, provide a score between 0.0 and 1.0.

```
=== INSTRUCTION ===
{Instruction}
```

```
=== HISTORY ===
{History}
```

```
=== NOW ===
{Current state}
```

### Zero-shot Prompt Template for WebShop

```
...(Preamble identical to WebShop value function template)
First, analyze the real task data provided below.

=== INSTRUCTION ===
{Instruction}

=== HISTORY ===
{History}

=== NOW ===
{Current state}

Your response MUST be a single floating-point number between 0.0 and 1.0, and
NOTHING ELSE.

Score:
```

### Zero-shot Prompt Template for SciWorld

```
...(Preamble identical to SciWorld value function template)
First, analyze the real task data provided below.

=== INSTRUCTION ===
{Instruction}

=== HISTORY ===
{History}

=== NOW ===
{Current state}

Your response MUST be a single floating-point number between -1.0 and 1.0, and
NOTHING ELSE.

Score:
```

### Zero-shot Prompt Template for TextCraft

```
...(Preamble identical to TextCraft value function template)
=== INSTRUCTION ===
{Instruction}

=== HISTORY ===
{History}

=== NOW ===
{Current state}

Your response MUST be a single floating-point number between 0.0 and 1.0, and
NOTHING ELSE.

Score:
```

### FoA Prompt Template for WebShop

Given an item to purchase and a trajectory that aims to buy an item that exactly matches the specification, which corresponds to the ideal score of 10, analyze the following trajectory, then at the last line conclude "Thus the correctness score is ", where score is an integer from 1 to 10.

Here are some examples

...(Same in-context examples as Klein et al. (2025))

### ReflAct Prompt Template for SciWorld

You are a helpful assistant to do some scientific experiment in an environment. In the environment, there are several rooms: kitchen, foundry, workshop, bathroom, outside, living room, bedroom, greenhouse, art studio, hallway. You should explore the environment and find the items you need to complete the experiment.

You can teleport to any room in one step.

All containers in the environment have already been opened, you can directly get items from the containers.

For each of your turn, you will be given the observation of the last turn.

You should first reflect on the agent's state, including the location, inventory, and focused object, in relation to the task goal.

Then, output the action for this turn. Your output must strictly follow this format:

Reflection:  
your reflection.

Action:  
your next action.

The available actions are:  
open OBJ: open a container  
close OBJ: close a container  
activate OBJ: activate a device  
deactivate OBJ: deactivate a device  
connect OBJ to OBJ: connect electrical components  
disconnect OBJ: disconnect electrical components  
use OBJ [on OBJ]: use a device/item  
look around: describe the current room  
examine OBJ: describe an object in detail  
look at OBJ: describe a container's contents  
read OBJ: read a note or book  
move OBJ to OBJ: move an object to a container  
pick up OBJ: move an object to the inventory  
pour OBJ into OBJ: pour a liquid into a container  
mix OBJ: chemically mix a container  
teleport to LOC: teleport to a specific room  
focus on OBJ: signal intent on a task object  
wait: task no action for 10 steps  
wait1: task no action for a step

Example:

...(Same in-context examples as Kim et al. (2025))

## D. Additional Experiments

### D.1. Resampling Steps

#### D.1.1. TASK-SPECIFIC FIXED-STEP RESAMPLING

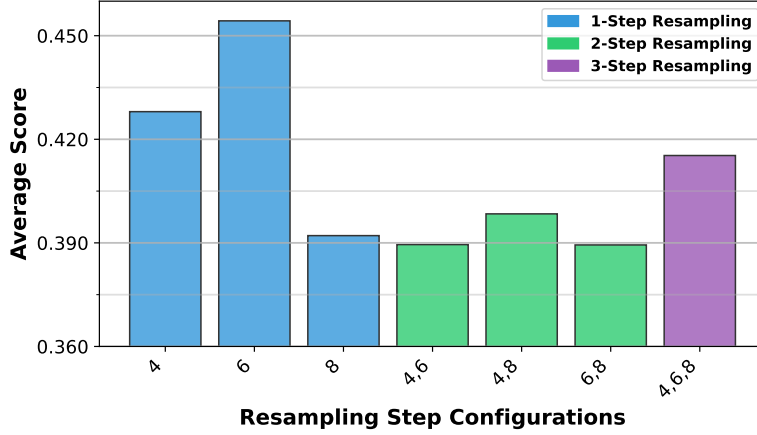


Figure 5. WebShop performance across different resampling steps.

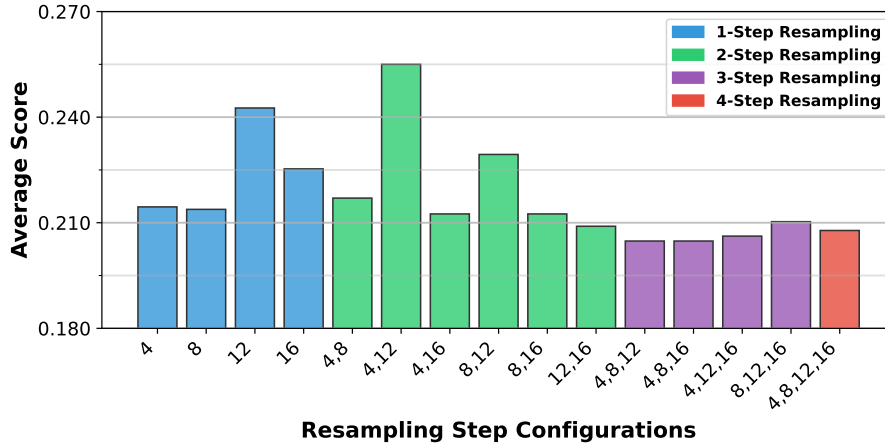


Figure 6. SciWorld performance across different resampling steps.

Our ablation study on resampling step configurations with 5 trajectories (Figure 5, Figure 6, and Figure 7) demonstrates that agent performance is highly adaptable to the strategic timing of value function interventions. Across all tested environments, results indicate that increasing resampling frequency does not inherently guarantee higher performance. Instead, AMC’s strength lies in its ability to provide high-impact, sparse corrections at pivotal moments. For example, in TextCraft, a dense 4-step resampling approach actually hindered performance compared to more focused 1-step or 2-step interventions. Peak efficiency is reached through task-specific timing: a single intervention at step 6 for WebShop, a dual-step approach at steps 4 and 12 for SciWorld, and an early correction at step 4 for TextCraft. These results highlight that AMC’s effectiveness is driven by precision-targeted interventions rather than sheer computational volume, allowing for significant performance gains while maintaining minimal overhead.

#### D.1.2. DYNAMIC RESAMPLING WITH EFFECTIVE SAMPLE SIZE

Empirical analysis in Appendix D.1.1 reveals that the optimal resampling frequency is inherently task-dependent. For instance, WebShop requires early-to-mid trajectory interventions to preserve the enough step budget to finalize transactions. To confirm this, we compare fixed-step resampling against a dynamic criterion based on the Effective Sample Size (ESS) (Doucet et al., 2000). Specifically, we resample at step  $t$  using normalized weights  $\tilde{w}_t^{(i)}$  whenever  $ESS :=$

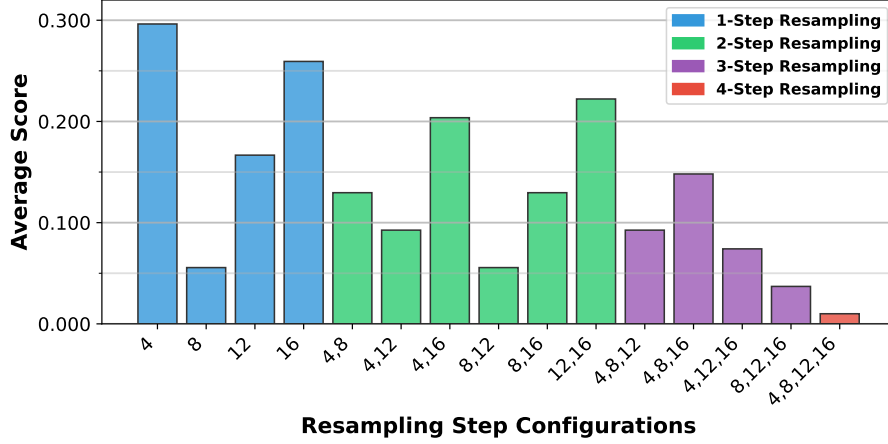


Figure 7. TextCraft performance across different resampling steps.

$(\sum_{i=1}^N (\tilde{w}_t^{(i)})^2)^{-1} < N\rho$  for a given threshold  $\rho \in (0, 1)$ . While ESS is designed to mitigate trajectory degeneracy by filtering low-weight samples, Table 8 shows that fixed-step selections consistently outperform ESS across various thresholds on both WebShop and SciWorld. Beyond superior performance, fixed-step resampling is significantly more computationally efficient. While ESS requires value network evaluation at every step, fixed-step allows the recursive update rule (Equation 4) to telescope, effectively skipping value estimation at most steps.

Table 8. Comparison of dynamic resampling and fixed-step resampling for AMC.

| Dataset  | Policy Model  | Method    | Resampling ( $\rho$ ) | Score                                 |
|----------|---------------|-----------|-----------------------|---------------------------------------|
| WebShop  | Llama-3.2-11B | Best-of-5 | -                     | 0.385 ( $\pm 0.015$ )                 |
|          |               | AMC       | ESS (0.1)             | 0.387 ( $\pm 0.015$ )                 |
|          |               |           | ESS (0.5)             | 0.402 ( $\pm 0.017$ )                 |
|          |               |           | ESS (0.9)             | <b>0.435 (<math>\pm 0.013</math>)</b> |
|          |               |           | Fixed-step            | <b>0.454 (<math>\pm 0.009</math>)</b> |
| SciWorld | Llama-3.1-8B  | Best-of-5 | -                     | 0.206 ( $\pm 0.010$ )                 |
|          |               | AMC       | ESS (0.1)             | 0.211 ( $\pm 0.007$ )                 |
|          |               |           | ESS (0.5)             | 0.216 ( $\pm 0.006$ )                 |
|          |               |           | ESS (0.9)             | 0.228 ( $\pm 0.006$ )                 |
|          |               |           | Fixed-step            | <b>0.255 (<math>\pm 0.008</math>)</b> |

 Table 9. Comparison of value estimation objectives when  $N = 15$ .

| Dataset  | Policy Model | Sampling   | Score                 |
|----------|--------------|------------|-----------------------|
| SciWorld | Llama-3.1-8B | Equation 7 | 0.347 ( $\pm 0.015$ ) |
|          |              | Equation 5 | 0.332 ( $\pm 0.005$ ) |

## D.2. Value Estimation Approaches

As discussed in Section 2.3, we approximate the ideal value  $V$  in Equation 5, which requires a soft maximum over multiple trajectories per task, with the single trajectory formulation in Equation 7. Multiple sampling has also been used in MCMC frameworks for uncertainty quantification in LLMs (Ross et al., 2026). To evaluate the empirical impact of this approximation bias, we compare the performance of both methods on SciWorld in Table 9. For Equation 5, we use 3 trajectory estimates to better approximate the log-sum-exp. We find that the single trajectory approximation yields comparable performance to the multiple trajectory baseline. This indicates that Equation 7 serves as an effective surrogate for Equation 5. In the context of our training set construction approach (Appendix B), in which we sample multiple trajectories per task and only include the max-reward trajectory, one can view Equation 7 as using a hard maximum to approximate the soft maximum of Equation 5.

### D.3. Impact of Base Model Selection on Value Function

We evaluated the AMC framework using an additional base model architecture for the value model, Qwen-3-4B, alongside open-weight policies, to verify that the value function’s effectiveness is not dependent on a specific base model from the Llama-3 family. As illustrated in Table 10, a Qwen-3-4B based value model achieves competitive performance across all benchmarks, closely trailing the results of the Llama-3 family. This confirms the architectural generalizability of the value function and suggests that overall performance can be further optimized by selecting the base model best suited to a specific benchmark.

Table 10. Sensitivity of the value function backbone across benchmarks with 5 trajectories.

| Dataset   | Policy Model  | Value Model   | Score                 |
|-----------|---------------|---------------|-----------------------|
| WebShop   | Llama-3.2-11B | Llama-3.2-11B | 0.454 ( $\pm 0.014$ ) |
|           |               | Qwen-3-4B     | 0.444 ( $\pm 0.017$ ) |
| SciWorld  | Llama-3.1-8B  | Llama-3.1-8B  | 0.249 ( $\pm 0.011$ ) |
|           |               | Qwen-3-4B     | 0.236 ( $\pm 0.011$ ) |
| TextCraft | Llama-3.2-11B | Llama-3.2-11B | 0.234 ( $\pm 0.012$ ) |
|           |               | Qwen-3-4B     | 0.222 ( $\pm 0.043$ ) |

### D.4. GRPO Baseline and Cost Analysis

Since Xi et al. (2025b) did not release their GRPO models, we used a 8xA100 40GB node to fine-tune our own GRPO baselines for 2 epochs, removing a leak of 30% of test tasks identified in the AgentGym-RL codebase. As shown in Table 11, GRPO with Best-of- $N$  provides only marginal improvements over GRPO across different policies, as the GRPO policy is already converged with a low standard error across 3 trials. When considering GPT-5.1 with a Qwen-2.5-7B value model, AMC significantly outperforms GRPO models with Best-of- $N$ , showing the practicality of implementing our approach on a black-box policy. When considering the same policy model as GRPO (i.e., Qwen-2.5-3B), we outperform GRPO at the same  $N$  while saving on total training and inference costs. Note that cost comparisons use AWS on-demand rates as of April 2026 (USD 21.96/hr for 8xA100, which is used only for GRPO training; USD 5.27/hr for an NVIDIA RTX 6000 96GB, which is the most comparable currently available server GPU to our main evaluation environment).

Table 11. Performance and cost efficiency comparison between GRPO and AMC across varying numbers of trajectories.

| Training | Method           | Policy Model | Value Model | Score                                 | Training Cost (USD) | Inference Cost (USD) |
|----------|------------------|--------------|-------------|---------------------------------------|---------------------|----------------------|
| GRPO     | ReAct            | Qwen-2.5-7B  | -           | 0.183 ( $\pm 0.001$ )                 | 702.64              | <b>10.54</b>         |
|          | Best-of-5        | Qwen-2.5-7B  | -           | 0.190 ( $\pm 0.001$ )                 | 702.64              | 55.32                |
|          | Best-of-20       |              |             | 0.194 ( $\pm 0.001$ )                 | 702.64              | 231.80               |
| -        | AMC ( $N = 5$ )  | GPT-5.1      | Qwen-2.5-7B | <b>0.518 (<math>\pm 0.030</math>)</b> | <b>138.39</b>       | 57.97                |
| GRPO     | ReAct            | Qwen-2.5-3B  | -           | 0.182 ( $\pm 0.001$ )                 | 395.24              | <b>5.27</b>          |
|          | Best-of-5        | Qwen-2.5-3B  | -           | 0.185 ( $\pm 0.003$ )                 | 395.24              | 12.28                |
|          | Best-of-20       |              |             | 0.187 ( $\pm 0.001$ )                 | 395.24              | 50.05                |
| -        | AMC ( $N = 5$ )  | Qwen-2.5-3B  | Qwen-2.5-3B | 0.133 ( $\pm 0.006$ )                 | 147.51              | 47.41                |
|          | AMC ( $N = 20$ ) |              |             | <b>0.216 (<math>\pm 0.004</math>)</b> | <b>147.51</b>       | 210.73               |

### D.5. Evaluating for Average Trajectory Performance

Our primary evaluation relies on the highest-reward trajectory. This is the most practical metric, as downstream applications typically deploy only the single best outcome. However, to ensure our results are robust against high-variance noise, we expand our evaluation to include an alternative metric. In Table 12, we evaluate the mean reward of the top 5 trajectories (out of 15 generated) for each task, averaged these results across all tasks in the dataset. Using Llama-based policies across WebShop, SciWorld, and TextCraft, AMC consistently outperforms the average top-5-of-15 baseline (derived from the Best-of-15). These results indicate that AMC significantly elevates the overall quality of the top-performing trajectories within the trajectory pool, rather than relying on isolated high-reward outliers to achieve a superior score. These results are

corroborated by the analysis in Appendix A.4, in which we show that, on expectation, AMC is guaranteed to improve the average reward across trajectories.

Table 12. Comparison of the averaged top 5 highest reward trajectories.

| Dataset   | Method             | Score                                 |
|-----------|--------------------|---------------------------------------|
| WebShop   | Best-of-15 (Top-5) | 0.202 ( $\pm$ 0.014)                  |
|           | AMC (Top-5)        | <b>0.371 (<math>\pm</math> 0.013)</b> |
| SciWorld  | Best-of-15 (Top-5) | 0.183 ( $\pm$ 0.010)                  |
|           | AMC (Top-5)        | <b>0.226 (<math>\pm</math> 0.014)</b> |
| TextCraft | Best-of-15 (Top-5) | 0.215 ( $\pm$ 0.030)                  |
|           | AMC (Top-5)        | <b>0.462 (<math>\pm</math> 0.020)</b> |

## D.6. Additional Datasets

To further evaluate the robustness of our approach, we test AMC on two additional tasks from the AgentBoard benchmark (Ma et al., 2024). Unlike our previous benchmarks, these datasets emphasize tool use, temporal synthesis, and multi-hop relational reasoning:

- **Weather:** This task evaluates the agent’s ability to perform multi-step information retrieval using a specialized Weather API. The agent navigates 18 distinct actions (e.g., querying historical data, local forecasts) to answer complex queries such as comparing precipitation across different time periods. Success requires precise tool-calling and temporal logic. We used a strict binary reward (1 only for an exact match with the ground truth), a maximum horizon of  $T = 10$ , and a resampling step of 4.
- **Movie:** This dataset tests the agent’s ability to perform relational queries within a large-scale cinematic database. With 16 available actions, the agent often executes multi-hop reasoning (e.g., identifying a specific director’s filmography, filtering by release year, and then retrieving the lead actor’s accolades). We utilized a strict binary reward (1 only for an exact match with the ground truth), a maximum horizon of  $T = 12$ , and a resampling step of 6.

As shown in Table 13, AMC consistently matches or outperforms the Best-of- $N$  and SMC (Zero-shot) baselines across these new domains. These results demonstrate that AMC’s performance gains are robust and generalizable to diverse tasks with varying requirements.

## E. Qualitative Analysis

Figure 8 provides a side-by-side comparison of two AMC trajectories on WebShop, highlighting a critical divergence between the two at resampling step 6. In Case 1, the agent identifies a product candidate ([B09LM3H2F5]) that initially satisfies several key semantic and budgetary criteria. This partial alignment yields a high value score of 0.6, which encourages the agent to proceed with this trajectory, ultimately securing a successful 1.0 reward through early task completion. In contrast, Case 2 illustrates a representative failure mode. While the agent initiates the task with the same search query as Case 1, it fails to prioritize relevant items, eventually returning to the initial search interface at step 6. At this step, our value model assigns a low score of 0.1, identifying the trajectory as being far from the target state, from which it goes on to achieve a reward of 0.0. This qualitative comparison confirms that our value function effectively distinguishes between goal-aligned progress and unproductive behavior at resampling step 6.

The value model also effectively distinguishes between promising and non-promising trajectories within the SciWorld environment. For the qualitative evaluation, we focus on resampling step 12, the second-most effective intervention from our ablation study (Figure 6), as it provides a clearer baseline for analyzing individual trajectory outcomes. As illustrated in Figure 9 (left), by step 12, the agent has successfully located its target—a non-living object (finger painting). At this stage, the value model assigns a high score of 0.4 out of its range of  $[-1, 1]$ . This trajectory culminates at step 15, where the agent enters the bathroom and places the painting in the red box, achieving a perfect environment reward of 1.0. Conversely, the non-promising trajectory in Figure 9 (right) illustrates the agent repeatedly performing redundant actions to access the bathroom despite the absence of a direct path from the hallway. This lack of progress results in a low value function score of

0.1 at step 12. Although the agent eventually identifies the target non-living item at step 18 and attempts to pick it up in the final step, it ultimately receives an environment reward of 0.0. This failure stems from the agent’s inability to satisfy necessary sub-goals, such as the initial “focus on” action required by the objective.

Furthermore, the value function can accurately provide more granular comparisons between relatively similar states. Figure 10 presents a qualitative analysis of two partial trajectories within the TextCraft domain. While both trajectories align with the optimal policy, Case 1 has progressed further along the crafting dependency tree compared to Case 2. Reflecting this state progression, the value function assigns a higher value estimate to the former (0.275) than the latter (0.191). This demonstrates that the value function can identify states closer to the terminal condition possess a higher probability of success.

Figure 8. Comparison of agent trajectories in WebShop. We highlight the divergence at the resampling step 6, where the value function differentiates between promising and non-promising ones.

| Case 1: Promising (Success)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Case 2: Non-Promising (Failure)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Goal:</b> Find me hand crafted candy &amp; chocolate for gift set, valentine day with style: happy birthday truffle chocolate, and price lower than 60.00 dollars.</p> <hr/> <p><b>[STEP 1]</b><br/> <b>Action:</b><br/>                     search[hand crafted candy gift set valentine day truffle chocolate birthday price:60]</p> <p>...(Search around items)</p> <p><b>[STEP 6]</b><br/> <b>Observation:</b> Item [B09LM3H2F5] partially matches gift set, truffle, and &lt; \$60.<br/> <b>Value Function: 0.6</b><br/> <b>Action:</b> click[B09LM3H2F5]</p> <p><b>[STEP 7]</b><br/> <b>Observation:</b> Options for style refinement: <i>happy birthday truffle chocolate, holiday truffle chocolate, truffle collection chocolate.</i><br/> <b>Action:</b> click[style]</p> <p><b>[STEP 8]</b><br/> <b>Observation:</b> Specific style options available.<br/> <b>Action:</b><br/>                     click[happy birthday truffle chocolate]</p> <p><b>[STEP 9]</b><br/> <b>Observation:</b> Identified product matching all criteria.<br/> <b>Action:</b> click[Buy Now]</p> <hr/> <p><b>Environment Reward: 1.0 (Success)</b></p> | <p><b>Goal:</b> Find me hand crafted candy &amp; chocolate for gift set, valentine day with style: happy birthday truffle chocolate, and price lower than 60.00 dollars.</p> <hr/> <p><b>[STEP 1]</b><br/> <b>Action:</b><br/>                     search[hand crafted candy chocolate gift set valentine happy birthday truffle price lower 60]</p> <p>...(Search around items)</p> <p><b>[STEP 6]</b><br/> <b>Observation:</b> Back to the first page to search.<br/> <b>Value Function: 0.1</b><br/> <b>Action:</b><br/>                     search[hand crafted candy chocolate gift set valentine happy birthday truffle price lower 60]</p> <p><b>[STEP 7]</b><br/> <b>Observation:</b> Search results for previous request.<br/> <b>Action:</b><br/>                     search[hand crafted candy chocolate gift set valentine happy birthday truffle chocolate gift box]</p> <p><b>[STEP 8]</b><br/> <b>Observation:</b> Search results for previous request.<br/> <b>Action:</b><br/>                     click[Waterbridge Wave Milk Chocolate Hazelnut Crunch]</p> <p><b>[STEP 9]</b><br/> <b>Observation:</b> View product page for Waterbridge Wave...<br/> <b>Action:</b> click[Add to Cart]</p> <p><b>[STEP 10]</b><br/> <b>Observation:</b> No state transition.<br/> <b>Action:</b><br/>                     click[CRAVINGS BY ZOE Valentine’s Day Chocolate Truffles]</p> <hr/> <p><b>Environment Reward: 0.0 (Failure)</b></p> |

Figure 9. Comparison of agent trajectories in SciWorld. We show the divergence at the resampling step 12, where the value function distinguishes between promising and non-promising ones.

| Case 1: Promising (Success)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Case 2: Non-Promising (Failure)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Goal:</b> Your task is to find a(n) non-living thing. First, focus on the thing. Then, move it to the red box in the bathroom.</p> <p>This room is called the hallway. In it, you see:<br/>the agent, a substance called air, a finger painting<br/>You also see:<br/>A door to the kitchen (that is closed)</p> <p>... (Multiple doors to alternative locations are visible, but a door for the bathroom is absent)</p> <hr/> <p><b>[STEP 1]</b><br/><b>Action:</b> Focus on the finger painting</p> <p><b>[STEP 2]</b><br/><b>Observation:</b> You focus on the finger painting.<br/><b>Action:</b> pick up the finger painting</p> <p>... (Explore the hallway)</p> <p><b>[STEP 9]</b><br/><b>Observation:</b> You move through the door to the kitchen.<br/><b>Action:</b> look around</p> <p><b>[STEP 10]</b><br/><b>Observation:</b> The description of the kitchen. There is a door to the bathroom (that is closed)<br/><b>Action:</b> go to LOC (bathroom)</p> <p><b>[STEP 11]</b><br/><b>Observation:</b> No known action matches that input.<br/><b>Action:</b> open bathroom door</p> <p><b>[STEP 12]</b><br/><b>Observation:</b> The door is now open.<br/><b>Value Function: 0.4</b><br/><b>Action:</b> look around</p> <p>... (Explore the kitchen)</p> <p><b>[STEP 14]</b><br/><b>Observation:</b> You move through the door to the bathroom.<br/><b>Action:</b> look around</p> <p><b>[STEP 15]</b><br/><b>Observation:</b> The description of the bathroom. There is a red box (containing nothing)<br/><b>Action:</b> move finger painting to red box</p> <hr/> <p><b>Environment Reward: 1.0 (Success)</b></p> | <p><b>Goal:</b> Your task is to find a(n) non-living thing. First, focus on the thing. Then, move it to the red box in the bathroom.</p> <p>This room is called the hallway. In it, you see:<br/>the agent, a substance called air, a finger painting<br/>You also see:<br/>A door to the kitchen (that is closed)</p> <p>... (Multiple doors to alternative locations are visible, but a door for the bathroom is absent)</p> <hr/> <p><b>[STEP 1]</b><br/><b>Action:</b> look around</p> <p><b>[STEP 2]</b><br/><b>Observation:</b> The description of the hallway.<br/><b>Action:</b> go to LOC (bathroom)</p> <p><b>[STEP 3]</b><br/><b>Observation:</b> No known action matches that input.<br/><b>Action:</b> go to LOC (bathroom)</p> <p>... (Redundant actions attempted to access the bathroom)</p> <p><b>[STEP 12]</b><br/><b>Observation:</b> No known action matches that input.<br/><b>Value Function: 0.1</b><br/><b>Action:</b> go to LOCATION bathroom</p> <p>... (Redundant actions attempted to access the bathroom)</p> <p><b>[STEP 17]</b><br/><b>Observation:</b> No known action matches that input.<br/><b>Action:</b> look around</p> <p><b>[STEP 18]</b><br/><b>Observation:</b> The description of the hallway.<br/><b>Action:</b> look at finger painting</p> <p><b>[STEP 19]</b><br/><b>Observation:</b> a finger painting of a cow. The artist is listed as Owen.<br/><b>Action:</b> move finger painting to bathroom</p> <p><b>[STEP 20]</b><br/><b>Observation:</b> No known action matches that input.<br/><b>Action:</b> pick up finger painting</p> <hr/> <p><b>Environment Reward: 0.0 (Failure)</b></p> |

Figure 10. Comparison of agent trajectories in TextCraft. We highlight the divergence at the resampling step 4, where the value function differentiates between more promising and less promising trajectories.

| Case 1: Promising (Success)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Case 2: Non-Promising (Failure)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Crafting commands:<br/> craft 1 lever using 1 cobblestone, 1 stick<br/> craft 6 andesite slab using 3 andesite<br/> craft 6 andesite wall using 6 andesite<br/> craft 6 cobblestone wall using 6 cobblestone<br/> craft 2 diorite using 2 quartz, 2 cobblestone<br/> craft 4 andesite stairs using 6 andesite<br/> craft 1 granite using 1 diorite, 1 quartz<br/> craft 1 mossy cobblestone using 1 cobblestone, 1 vine<br/> craft 1 quartz block using 4 quartz<br/> craft 1 dropper using 1 redstone, 7 cobblestone<br/> craft 1 dispenser using 1 redstone, 7 cobblestone, 1 bow<br/> craft 4 diorite stairs using 6 diorite<br/> craft 2 andesite using 1 diorite, 1 cobblestone<br/> <b>Goal:</b> craft andesite wall.</p> <hr/> <p><b>[STEP 1]</b><br/> <b>Action:</b><br/> Inventory</p> <p><b>[STEP 2]</b><br/> <b>Observation:</b><br/> Inventory: You are not carrying anything.<br/> <b>Action:</b><br/> Get 2 quartz<br/> Get 2 cobblestone</p> <p><b>[STEP 3]</b><br/> <b>Observation:</b><br/> Got 2 quartz.<br/> <b>Action:</b><br/> Get 2 cobblestone</p> <p><b>[STEP 4]</b><br/> <b>Observation:</b><br/> Got 2 cobblestone.<br/> <b>Value Function: 0.275</b><br/> <b>Action:</b><br/> craft 2 diorite using 2 quartz, 2 cobblestone<br/> ...</p> <p><b>[STEP 16]</b><br/> <b>Observation:</b><br/> Could not execute [no action].</p> <p><b>Action:</b><br/> craft 6 andesite wall using 6 andesite</p> <hr/> <p><b>Environment Reward: 1.0 (Success)</b></p> | <p>Crafting commands:<br/> craft 1 lever using 1 cobblestone, 1 stick<br/> craft 6 andesite slab using 3 andesite<br/> craft 6 andesite wall using 6 andesite<br/> craft 6 cobblestone wall using 6 cobblestone<br/> craft 2 diorite using 2 quartz, 2 cobblestone<br/> craft 4 andesite stairs using 6 andesite<br/> craft 1 granite using 1 diorite, 1 quartz<br/> craft 1 mossy cobblestone using 1 cobblestone, 1 vine<br/> craft 1 quartz block using 4 quartz<br/> craft 1 dropper using 1 redstone, 7 cobblestone<br/> craft 1 dispenser using 1 redstone, 7 cobblestone, 1 bow<br/> craft 4 diorite stairs using 6 diorite<br/> craft 2 andesite using 1 diorite, 1 cobblestone<br/> <b>Goal:</b> craft andesite wall.</p> <hr/> <p><b>[STEP 1]</b><br/> <b>Action:</b><br/> Inventory</p> <p><b>[STEP 2]</b><br/> <b>Observation:</b><br/> Inventory: You are not carrying anything.<br/> <b>Action:</b><br/> get 2 diorite</p> <p><b>[STEP 3]</b><br/> <b>Observation:</b><br/> Could not find diorite.<br/> <b>Action:</b><br/> Get 2 quartz</p> <p><b>[STEP 4]</b><br/> <b>Observation:</b><br/> Got 2 quartz.<br/> <b>Value Function: 0.191</b><br/> <b>Action:</b><br/> Get 2 cobblestone<br/> ...</p> <p><b>[STEP 20]</b><br/> <b>Observation:</b><br/> Crafted 2 minecraft:andesite.<br/> <b>Action:</b> get 1 diorite</p> <hr/> <p><b>Environment Reward: 0.0 (Failure)</b></p> |

Table 13. Weather and Movie performance, obtained with a Llama-3.2-11B-based value model for SMC (Zero-shot) and AMC.

| Dataset | Policy Model  | Method          | Score                                 |
|---------|---------------|-----------------|---------------------------------------|
| Weather | Llama-3.2-11B | ReAct           | 0.433 ( $\pm 0.041$ )                 |
|         |               | Best-of-5       | <b>0.517 (<math>\pm 0.020</math>)</b> |
|         |               | SMC (Zero-shot) | <b>0.483 (<math>\pm 0.054</math>)</b> |
|         |               | AMC             | <b>0.533 (<math>\pm 0.020</math>)</b> |
|         |               | Best-of-15      | <b>0.550 (<math>\pm 0.000</math>)</b> |
|         |               | SMC (Zero-shot) | <b>0.550 (<math>\pm 0.000</math>)</b> |
|         |               | AMC             | <b>0.550 (<math>\pm 0.000</math>)</b> |
|         | GPT-4.1-mini  | ReAct           | 0.450 ( $\pm 0.035$ )                 |
|         |               | Best-of-5       | 0.483 ( $\pm 0.041$ )                 |
|         |               | SMC (Zero-shot) | 0.517 ( $\pm 0.020$ )                 |
|         |               | AMC             | <b>0.550 (<math>\pm 0.000</math>)</b> |
|         |               | Best-of-15      | <b>0.533 (<math>\pm 0.020</math>)</b> |
|         |               | SMC (Zero-shot) | <b>0.550 (<math>\pm 0.000</math>)</b> |
|         |               | AMC             | <b>0.550 (<math>\pm 0.000</math>)</b> |
|         | GPT-5.1       | ReAct           | 0.550 ( $\pm 0.000$ )                 |
|         |               | Best-of-5       | <b>0.550 (<math>\pm 0.000</math>)</b> |
|         |               | SMC (Zero-shot) | <b>0.550 (<math>\pm 0.000</math>)</b> |
|         |               | AMC             | <b>0.567 (<math>\pm 0.020</math>)</b> |
|         |               | Best-of-15      | 0.550 ( $\pm 0.000$ )                 |
|         |               | SMC (Zero-shot) | 0.550 ( $\pm 0.000$ )                 |
|         |               | AMC             | <b>0.583 (<math>\pm 0.020</math>)</b> |
| Movie   | Llama-3.2-11B | ReAct           | 0.417 ( $\pm 0.041$ )                 |
|         |               | Best-of-5       | <b>0.833 (<math>\pm 0.041</math>)</b> |
|         |               | SMC (Zero-shot) | <b>0.767 (<math>\pm 0.054</math>)</b> |
|         |               | AMC             | <b>0.783 (<math>\pm 0.020</math>)</b> |
|         |               | Best-of-15      | 0.850 ( $\pm 0.000$ )                 |
|         |               | SMC (Zero-shot) | <b>0.850 (<math>\pm 0.035</math>)</b> |
|         |               | AMC             | <b>0.883 (<math>\pm 0.020</math>)</b> |
|         | GPT-4.1-mini  | ReAct           | 0.750 ( $\pm 0.061$ )                 |
|         |               | Best-of-5       | 0.800 ( $\pm 0.035$ )                 |
|         |               | SMC (Zero-shot) | 0.817 ( $\pm 0.020$ )                 |
|         |               | AMC             | <b>0.850 (<math>\pm 0.000</math>)</b> |
|         |               | Best-of-15      | <b>0.850 (<math>\pm 0.000</math>)</b> |
|         |               | SMC (Zero-shot) | <b>0.850 (<math>\pm 0.000</math>)</b> |
|         |               | AMC             | <b>0.850 (<math>\pm 0.000</math>)</b> |
|         | GPT-5.1       | ReAct           | 0.867 ( $\pm 0.020$ )                 |
|         |               | Best-of-5       | <b>0.883 (<math>\pm 0.020</math>)</b> |
|         |               | SMC (Zero-shot) | 0.867 ( $\pm 0.020$ )                 |
|         |               | AMC             | <b>0.900 (<math>\pm 0.000</math>)</b> |
|         |               | Best-of-15      | <b>0.883 (<math>\pm 0.020</math>)</b> |
|         |               | SMC (Zero-shot) | <b>0.900 (<math>\pm 0.000</math>)</b> |
|         |               | AMC             | <b>0.900 (<math>\pm 0.000</math>)</b> |